

Genome Assembly

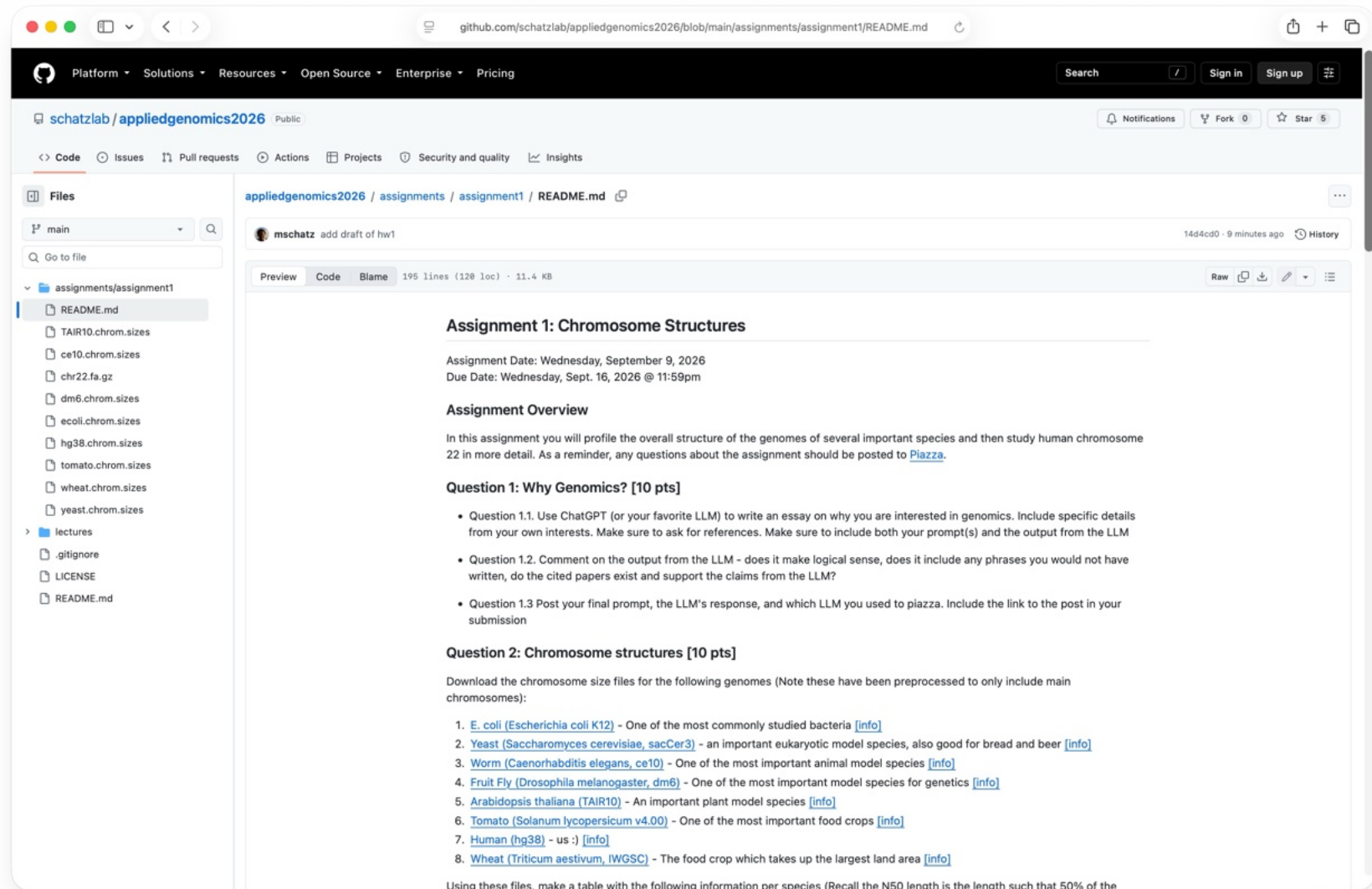
Michael Schatz

Sept 9, 2026

Lecture 3: Applied Comparative Genomics



Assignment I



github.com/schatzlab/appliedgenomics2026/blob/main/assignments/assignment1/README.md

Platform Solutions Resources Open Source Enterprise Pricing

Search Sign in Sign up

Notifications Fork 0 Star 5

Code Issues Pull requests Actions Projects Security and quality Insights

Files

main

Go to file

assignments/assignment1

- README.md
- TAIR10.chrom.sizes
- ce10.chrom.sizes
- chr22.fa.gz
- dm6.chrom.sizes
- ecoli.chrom.sizes
- hg38.chrom.sizes
- tomato.chrom.sizes
- wheat.chrom.sizes
- yeast.chrom.sizes

lectures

- .gitignore
- LICENSE
- README.md

appliedgenomics2026 / assignments / assignment1 / README.md

mschatz add draft of hw1 14d4cd0 · 9 minutes ago History

Preview Code Blame 195 lines (120 loc) · 11.4 KB Raw

Assignment 1: Chromosome Structures

Assignment Date: Wednesday, September 9, 2026
Due Date: Wednesday, Sept. 16, 2026 @ 11:59pm

Assignment Overview

In this assignment you will profile the overall structure of the genomes of several important species and then study human chromosome 22 in more detail. As a reminder, any questions about the assignment should be posted to [Piazza](#).

Question 1: Why Genomics? [10 pts]

- Question 1.1. Use ChatGPT (or your favorite LLM) to write an essay on why you are interested in genomics. Include specific details from your own interests. Make sure to ask for references. Make sure to include both your prompt(s) and the output from the LLM
- Question 1.2. Comment on the output from the LLM - does it make logical sense, does it include any phrases you would not have written, do the cited papers exist and support the claims from the LLM?
- Question 1.3 Post your final prompt, the LLM's response, and which LLM you used to piazza. Include the link to the post in your submission

Question 2: Chromosome structures [10 pts]

Download the chromosome size files for the following genomes (Note these have been preprocessed to only include main chromosomes):

- [E. coli \(Escherichia coli K12\)](#) - One of the most commonly studied bacteria [\[info\]](#)
- [Yeast \(Saccharomyces cerevisiae, sacCer3\)](#) - an important eukaryotic model species, also good for bread and beer [\[info\]](#)
- [Worm \(Caenorhabditis elegans, ce10\)](#) - One of the most important animal model species [\[info\]](#)
- [Fruit Fly \(Drosophila melanogaster, dm6\)](#) - One of the most important model species for genetics [\[info\]](#)
- [Arabidopsis thaliana \(TAIR10\)](#) - An important plant model species [\[info\]](#)
- [Tomato \(Solanum lycopersicum v4.00\)](#) - One of the most important food crops [\[info\]](#)
- [Human \(hg38\)](#) - us :) [\[info\]](#)
- [Wheat \(Triticum aestivum, IWGSC\)](#) - The food crop which takes up the largest land area [\[info\]](#)

Using these files, make a table with the following information per species (Recall the N50 length is the length such that 50% of the

<https://github.com/schatzlab/appliedgenomics2026/tree/main/assignments/assignment1>
Due end of day on Wednesday Sept 16 (right before midnight)

Piazza

The screenshot shows a web browser displaying a Piazza class page. The URL in the address bar is <https://piazza.com/class/mt9gq0k8adb77/post/11>. The page header includes the Piazza logo, the class ID '600.449/600.649', and navigation links for 'Q & A', 'Resources', 'Statistics', and 'Manage Class'. There are also buttons for 'Buy a License' and 'Switch to contribution model', and a user profile for 'Michael Schatz'.

The main content area is titled 'question @11' and 'HW1 Essay on Genomics'. It was updated 33 seconds ago by Michael Schatz. The post text reads: 'Reply to this post with your prompts and essays below. Here is my prompt and response with ChatGPT 6 Astra Medium: Write an essay about how genomics can be used to understand how changes in the genome are related to changes in my traits (genotype to phenotype). Make sure to include specific examples of some of the traits that have known genetics and some examples where it remains a mystery. This should make reference of some of my recent papers. Also include citations throughout the essay in the style of the journal Nature'.

The post includes a 'Response:' section with the following text: 'From genome to traits: what our DNA can explain—and what remains a mystery'.

What could reading my genome tell me about myself? It might help explain my blood type, my ability to digest milk, or some of my susceptibility to disease. It could also reveal thousands of variants associated with traits such as height. Yet even a complete genome sequence would leave many questions unanswered. Understanding the relationship between genotype—the DNA sequence we carry—and phenotype—our measurable characteristics—requires more than identifying genetic differences. We must discover how those differences influence molecules, cells, tissues and development, and how their effects depend on the environments in which we live. Genomics makes these connections by combining DNA sequencing with measurements of gene activity, comparisons among individuals and experiments that test biological mechanisms.¹

A genetic change can influence a trait in several ways. A substitution in a protein-coding sequence can alter the protein's activity. A change in a regulatory element can affect when, where or how strongly a gene is expressed. Larger changes, including deletions, duplications and rearrangements, can alter entire genes or their surrounding regulatory sequences. The consequences therefore depend on both the variant and its biological context. The Genotype-Tissue Expression project demonstrated the importance of this context by mapping genetic effects on gene expression and RNA splicing across human tissues. A variant's influence may be apparent in one tissue and much weaker in another, making the relevant cell type an essential part of the genotype-to-phenotype explanation.¹

Some traits have relatively direct genetic explanations. ABO blood type is a classic example. The ABO gene encodes an enzyme that modifies carbohydrate molecules on cell surfaces. Common A and B alleles encode enzymes with different biochemical specificities, whereas a common O allele contains a deletion that prevents the enzyme from functioning. These molecular differences explain the familiar A, B, AB and O blood groups. Although additional genes and uncommon variants can complicate the picture, ABO illustrates a remarkably clear chain from DNA sequence to protein function to an observable trait.²

The ability to digest lactose in adulthood provides a different lesson. Here, important variants affect gene regulation rather than the structure of a digestive enzyme. Lactase breaks down lactose, the sugar in milk, but its production normally decreases after childhood. Variants in regulatory DNA near *LCT*, the gene encoding lactase, can help maintain expression into adulthood. Research in African and European populations identified distinct variants associated with lactase persistence, including variants that increased transcription in functional assays. The same broad phenotype therefore evolved through different genetic changes. This example also shows why a genetic explanation discovered in one population may be incomplete when applied to another.³

Sickle haemoglobin offers an especially striking example of a small molecular change with major biological consequences. The sickle variant of *HBB* replaces a glutamic acid with valine in the β -globin protein. This alters haemoglobin's physical properties and helps explain why red blood cells can become distorted under low-oxygen conditions. Ingram's work identifying the chemical difference between normal and sickle haemoglobin established a foundational connection between inherited variation and altered protein structure. It showed that understanding a phenotype can require following the consequences of a genetic change down to a single amino acid.⁴

Most familiar human traits, however, do not follow such a simple pattern. Height is influenced by many genetic variants, each typically contributing a small effect, together with nutrition, health

The post has 0 likes and 0 views. There are buttons for 'Edit', 'Like', 'Bookmark', 'Star', and 'Link'.

<https://piazza.com/jhu/fall2026/600449600649>

TA: Nicole Brown



Office Hours: Wednesday @ 4:30pm
@ Malone 216 and Zoom (see Piazza)

Plotting in Python



<https://matplotlib.org/>

Plotting in R / ggplot2

Data visualization with ggplot2 : : CHEATSHEET

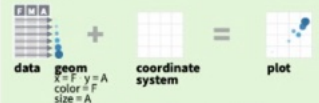


Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and **geoms**—visual marks that represent data points.



To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Complete the template below to build a graph.

```
ggplot(data = <DATA>) +  
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>),  
    stat = <STAT>, position = <POSITION>) +  
  <COORDINATE_FUNCTION> +  
  <FACET_FUNCTION> +  
  <SCALE_FUNCTION> +  
  <THEME_FUNCTION>
```

ggplot(data = mpg, aes(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

last_plot() Returns the last plot.

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5" x 5" file named "plot.png" in working directory. Matches file type to file extension.

Aes

Common aesthetic values.

color and **fill** - string ("red", "#RRGGBB")

linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dottedash", 5 = "longdash", 6 = "twodash")

size - integer (line width in mm)

shape - integer/shape name or a single character ("a")



Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

```
a <- ggplot(economics, aes(date, unemployment))  
b <- ggplot(seals, aes(x = long, y = lat))
```

a + geom_blank() and **a + expand_limits()**
Ensure limits include values across all plots.

b + geom_curve()
x, y, alpha, color, group, curvature, linetype, size

a + geom_path()
linejoin = "butt", linejoin = "round", linemitre = 1
x, y, alpha, color, group, linetype, size

a + geom_polygon()
aes(alpha = 50) - x, y, alpha, color, fill, group, subgroup, linetype, size

b + geom_rect()
aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon()
aes(ymin = unemployment - 900, ymax = unemployment + 900) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

```
b + geom_abline(aes(intercept = 0, slope = 1))  
b + geom_hline(aes(yintercept = lat))  
b + geom_vline(aes(xintercept = long))
```

```
b + geom_segment(aes(yend = lat + 1, xend = long + 1))  
b + geom_spoke(aes(angle = 1:1155, radius = 1))
```

ONE VARIABLE continuous

```
c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)
```

c + geom_area()
stat = "bin"
x, y, alpha, color, fill, linetype, size

c + geom_density()
kernel = "gaussian"
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly()
x, y, alpha, color, group, linetype, size

c + geom_histogram()
binwidth = 5
x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq()
aes(sample = hwy)
x, y, alpha, color, fill, linetype, size, weight

discrete

```
d <- ggplot(mpg, aes(fit))
```

d + geom_bar()
x, alpha, color, fill, linetype, size, weight

TWO VARIABLES both continuous

```
e <- ggplot(mpg, aes(cty, hwy))
```

e + geom_label()
aes(label = cty, nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_point()
x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile()
x, y, alpha, color, group, linetype, size, weight

e + geom_rug()
sides = "bl"
x, y, alpha, color, linetype, size

e + geom_smooth()
method = "lm"
x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text()
aes(label = cty, nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

one discrete, one continuous

```
f <- ggplot(mpg, aes(class, hwy))
```

f + geom_col()
x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot()
x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot()
binaxis = "y", stackdir = "center"
x, y, alpha, color, fill, group

f + geom_violin()
scale = "area"
x, y, alpha, color, fill, group, linetype, size, weight

both discrete

```
g <- ggplot(diamonds, aes(cut, color))
```

g + geom_count()
x, y, alpha, color, fill, shape, size, stroke

e + geom_jitter()
height = 2, width = 2
x, y, alpha, color, fill, shape, size

THREE VARIABLES

```
sealsSz <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))
```

l + geom_contour()
aes(z = z)
x, y, z, alpha, color, group, linetype, size, weight

l + geom_contour_filled()
aes(fill = z)
x, y, alpha, color, fill, group, linetype, size, subgroup

continuous bivariate distribution

```
h <- ggplot(diamonds, aes(carat, price))
```

h + geom_bin2d()
binwidth = c(0.25, 500)
x, y, alpha, color, fill, linetype, size, weight

h + geom_density_2d()
x, y, alpha, color, group, linetype, size

h + geom_hex()
x, y, alpha, color, fill, size

continuous function

```
i <- ggplot(economics, aes(date, unemployment))
```

i + geom_area()
x, y, alpha, color, fill, linetype, size

i + geom_line()
x, y, alpha, color, group, linetype, size

i + geom_step()
direction = "hv"
x, y, alpha, color, group, linetype, size

visualizing error

```
df <- data.frame(grp = c("A", "B"), fit = 4.5, se = 1.2)  
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))
```

j + geom_crossbar()
fatten = 2 - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar()
x, y, lower, upper, ymax, ymin, alpha, color, group, linetype, size, width
Also **geom_errorbarh()**.

j + geom_linerange()
x, y, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange()
x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

```
data <- data.frame(murder = USArrests$Murder,  
  state = tolower(row.names(USArrests)))
```

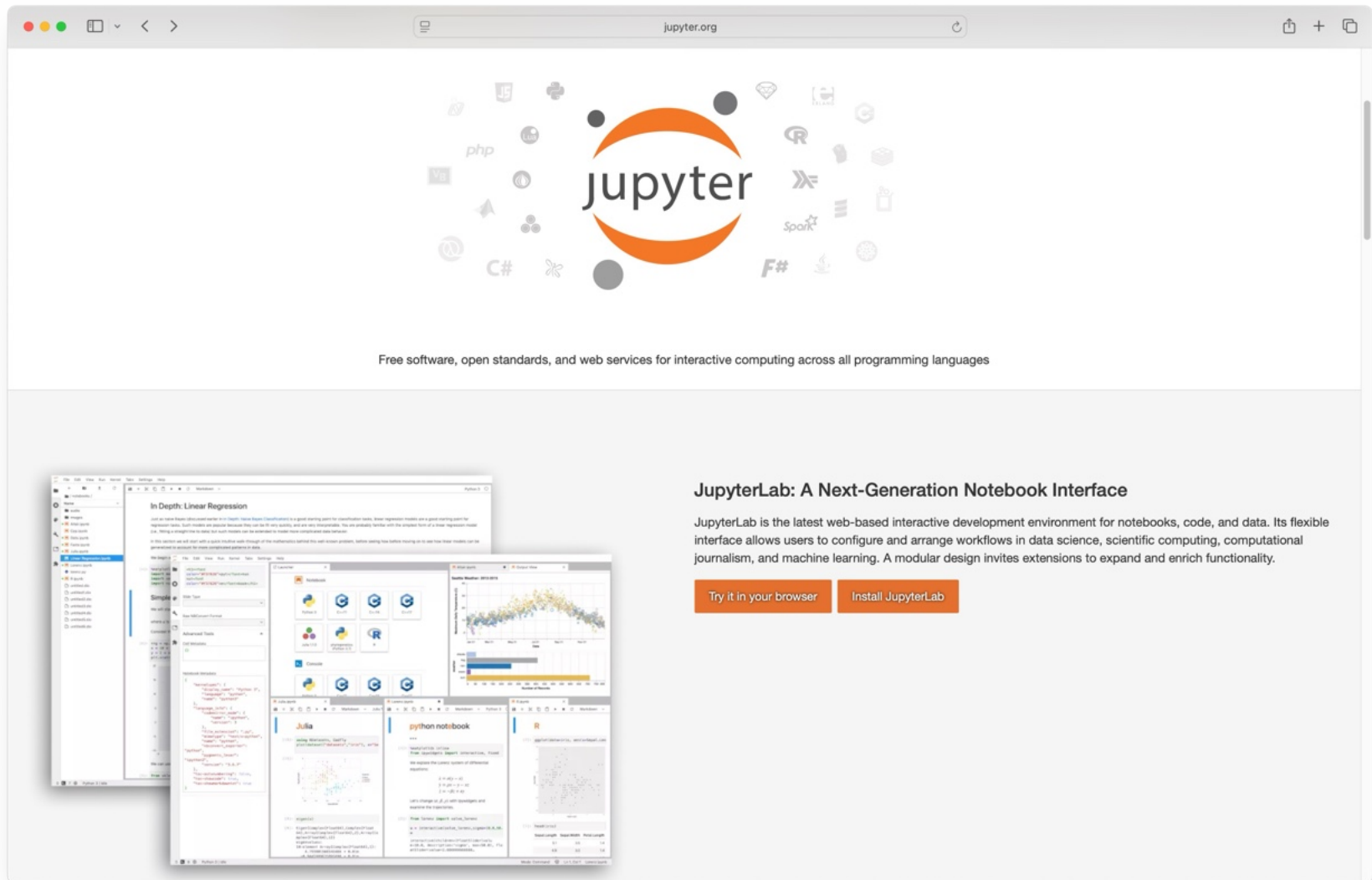
```
map <- map_data("state")  
k <- ggplot(data, aes(fill = murder))
```

k + geom_map()
aes(map_id = state), map = map
+ **expand_limits**(x = map\$long, y = map\$lat)
map_id, alpha, color, fill, linetype, size

l + geom_raster()
aes(fill = z), hjust = 0.5, vjust = 0.5, interpolate = FALSE
x, y, alpha, fill

l + geom_tile()
aes(fill = z)
x, y, alpha, color, fill, linetype, size, width

Jupyter Notebooks



The image shows a screenshot of the Jupyter.org website. The main heading is "Jupyter" with a logo consisting of a stylized orange 'J' and the word "jupyter" in lowercase. Below the logo, it says "Free software, open standards, and web services for interactive computing across all programming languages". The website also features a grid of logos for various programming languages and tools, including PHP, C#, F#, R, Spark, and others. Below the main heading, there is a section titled "JupyterLab: A Next-Generation Notebook Interface". This section describes JupyterLab as the latest web-based interactive development environment for notebooks, code, and data. It mentions that its flexible interface allows users to configure and arrange workflows in data science, scientific computing, computational journalism, and machine learning. At the bottom of this section, there are two buttons: "Try it in your browser" and "Install JupyterLab". Below the text, there is a screenshot of the JupyterLab interface, which shows a multi-paneled view with a file browser, a code editor, and several notebooks. One notebook is titled "In Depth: Linear Regression" and shows a scatter plot and a linear regression line. Another notebook is titled "python notebook" and shows a scatter plot. The JupyterLab interface is designed to be flexible and customizable, allowing users to arrange their workspace to suit their needs.

jupyter.org

Free software, open standards, and web services for interactive computing across all programming languages

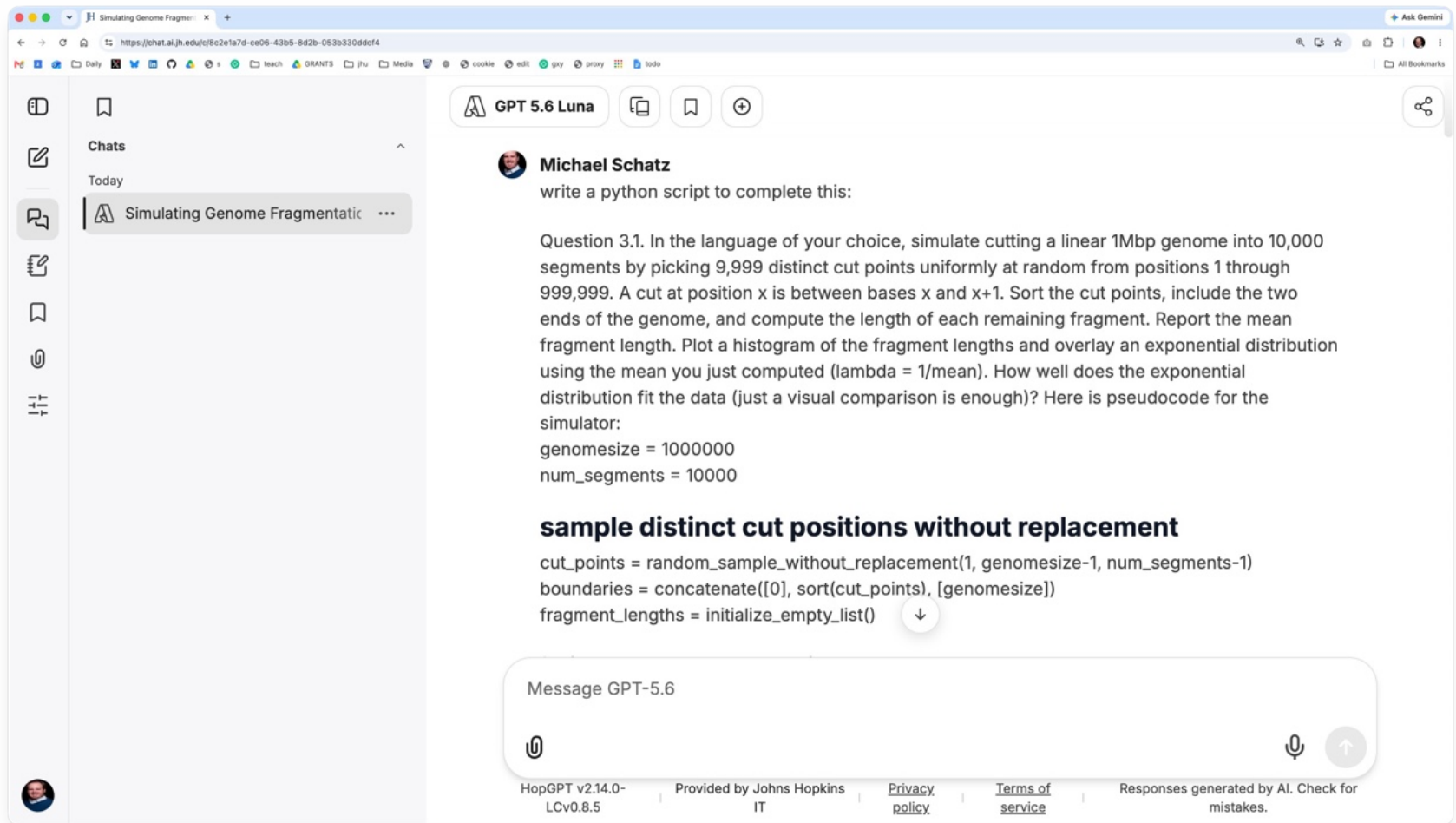
JupyterLab: A Next-Generation Notebook Interface

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science, scientific computing, computational journalism, and machine learning. A modular design invites extensions to expand and enrich functionality.

Try it in your browser Install JupyterLab

<https://jupyter.org>

HopGPT



The screenshot displays the HopGPT web interface. On the left is a sidebar with a 'Chats' section containing a chat titled 'Simulating Genome Fragmentation'. The main area shows a chat with 'GPT 5.6 Luna'. The user, Michael Schatz, has asked for a Python script to complete a task. The AI has provided a detailed response including a question description, pseudocode, and a code snippet. The interface includes a message input field at the bottom and a footer with version information and links.

GPT 5.6 Luna

Michael Schatz
write a python script to complete this:

Question 3.1. In the language of your choice, simulate cutting a linear 1Mbp genome into 10,000 segments by picking 9,999 distinct cut points uniformly at random from positions 1 through 999,999. A cut at position x is between bases x and $x+1$. Sort the cut points, include the two ends of the genome, and compute the length of each remaining fragment. Report the mean fragment length. Plot a histogram of the fragment lengths and overlay an exponential distribution using the mean you just computed ($\lambda = 1/\text{mean}$). How well does the exponential distribution fit the data (just a visual comparison is enough)? Here is pseudocode for the simulator:

```
genomesize = 1000000
num_segments = 10000
```

sample distinct cut positions without replacement

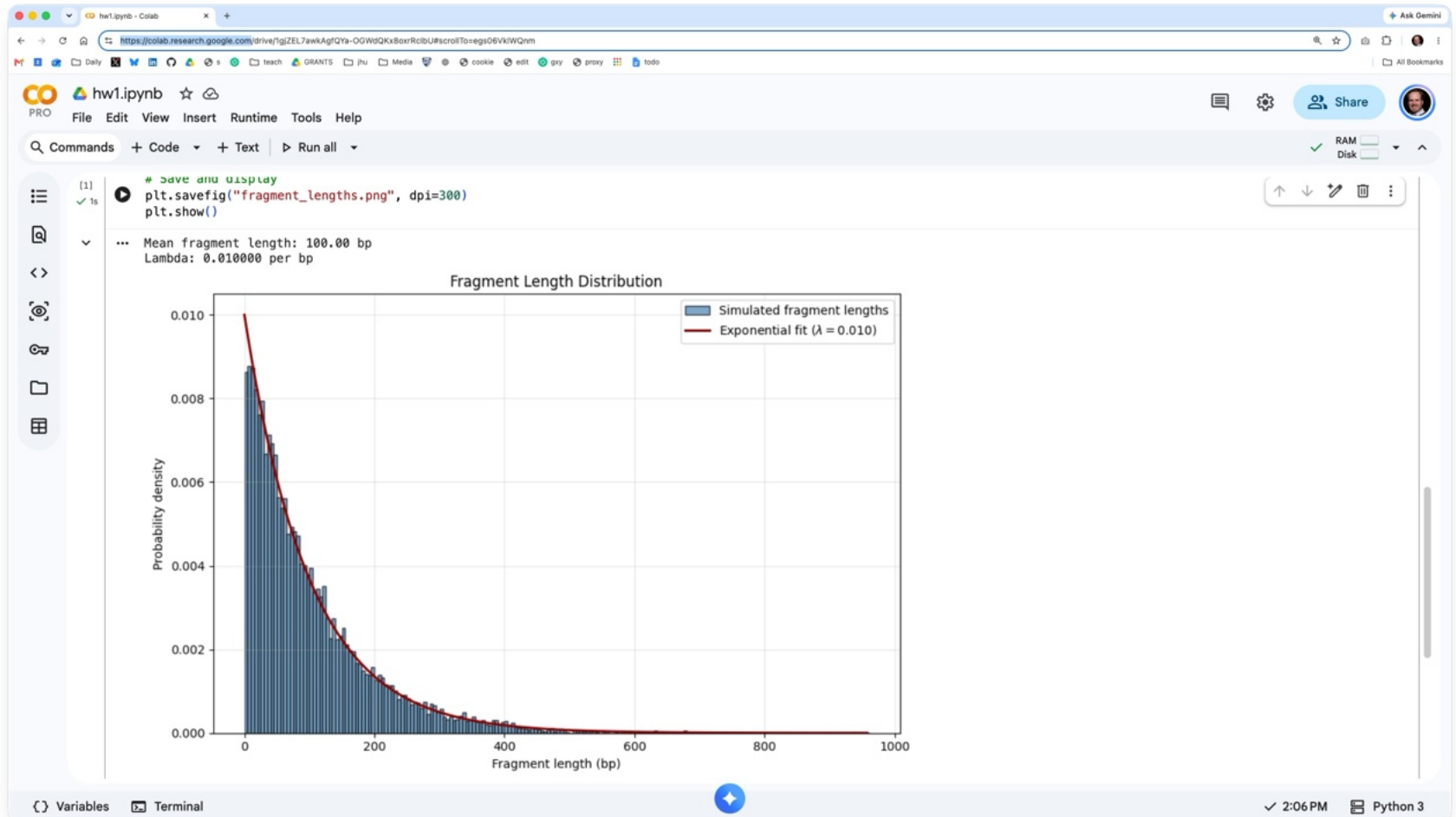
```
cut_points = random_sample_without_replacement(1, genomesize-1, num_segments-1)
boundaries = concatenate([0], sort(cut_points), [genomesize])
fragment_lengths = initialize_empty_list()
```

Message GPT-5.6

HopGPT v2.14.0-LCv0.8.5 | Provided by Johns Hopkins IT | [Privacy policy](#) | [Terms of service](#) | Responses generated by AI. Check for mistakes.

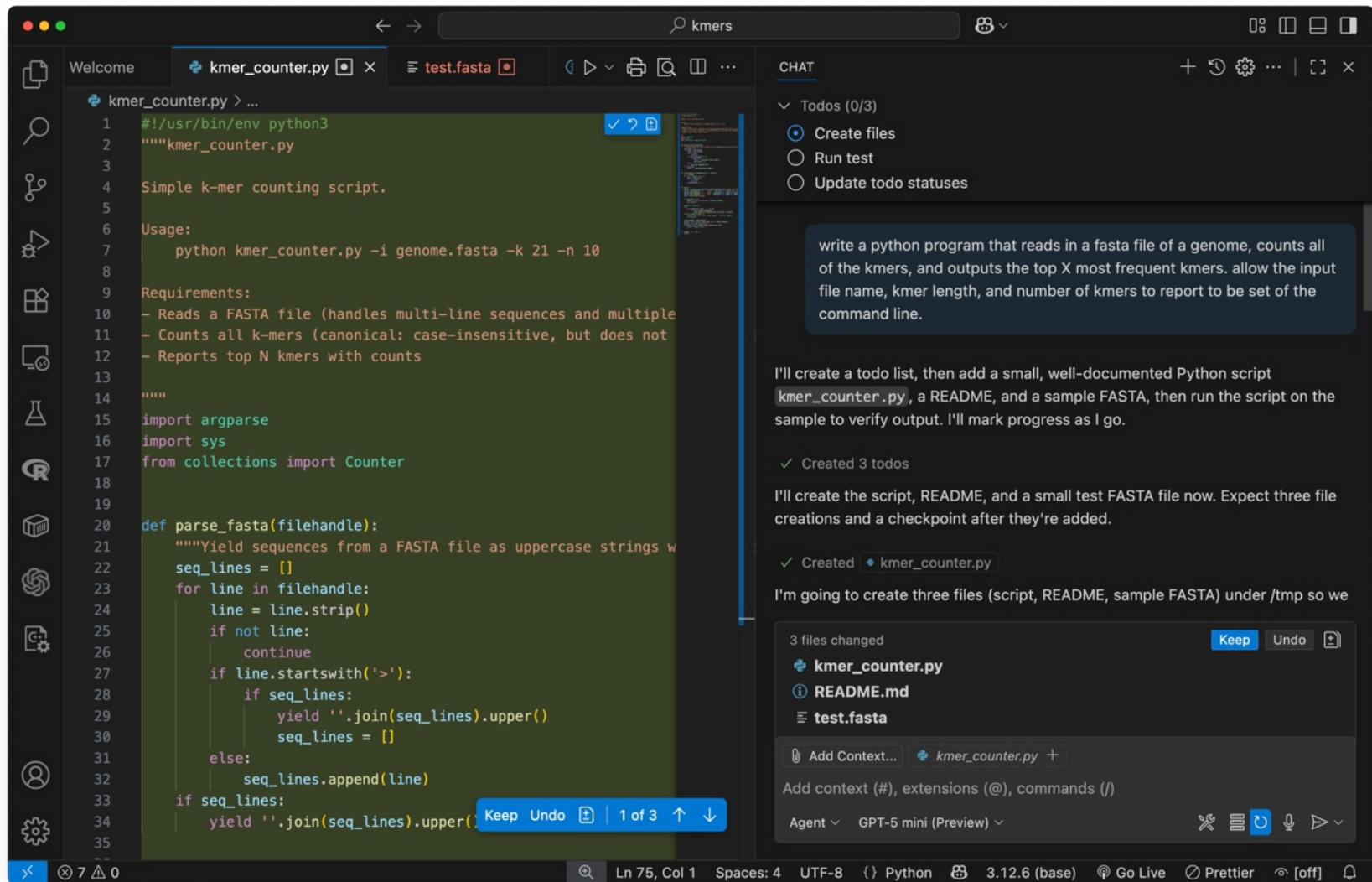
<https://chat.ai.jh.edu/>

Google Colab



<https://colab.research.google.com/>

VS Code



The screenshot displays the Visual Studio Code (VS Code) editor interface. The main editor window shows a Python script named `kmer_counter.py` with the following content:

```
1  #!/usr/bin/env python3
2  """kmer_counter.py
3
4  Simple k-mer counting script.
5
6  Usage:
7      python kmer_counter.py -i genome.fasta -k 21 -n 10
8
9  Requirements:
10     - Reads a FASTA file (handles multi-line sequences and multiple
11     - Counts all k-mers (canonical: case-insensitive, but does not
12     - Reports top N kmers with counts
13
14     """
15     import argparse
16     import sys
17     from collections import Counter
18
19
20     def parse_fasta(filehandle):
21         """Yield sequences from a FASTA file as uppercase strings w
22         seq_lines = []
23         for line in filehandle:
24             line = line.strip()
25             if not line:
26                 continue
27             if line.startswith('>'):
28                 if seq_lines:
29                     yield ''.join(seq_lines).upper()
30                     seq_lines = []
31             else:
32                 seq_lines.append(line)
33         if seq_lines:
34             yield ''.join(seq_lines).upper()
35
```

The right sidebar shows the CHAT panel with a list of todos (0/3) and a chat window. The chat window contains the following text:

write a python program that reads in a fasta file of a genome, counts all of the kmers, and outputs the top X most frequent kmers. allow the input file name, kmer length, and number of kmers to report to be set of the command line.

I'll create a todo list, then add a small, well-documented Python script `kmer_counter.py`, a README, and a sample FASTA, then run the script on the sample to verify output. I'll mark progress as I go.

✓ Created 3 todos

I'll create the script, README, and a small test FASTA file now. Expect three file creations and a checkpoint after they're added.

✓ Created `kmer_counter.py`

I'm going to create three files (script, README, sample FASTA) under /tmp so we

3 files changed Keep Undo

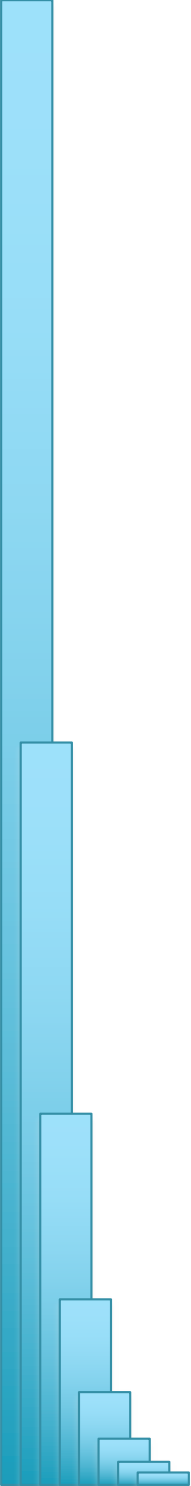
- `kmer_counter.py`
- `README.md`
- `test.fasta`

Add Context... `kmer_counter.py` +

Add context (#), extensions (@), commands (/)

Agent GPT-5 mini (Preview)

“write a python program that reads in a fasta file of a genome, counts all of the kmers, and outputs the top X most frequent kmers. allow the input file name, kmer length, and number of kmers to report to be set of the command line.”



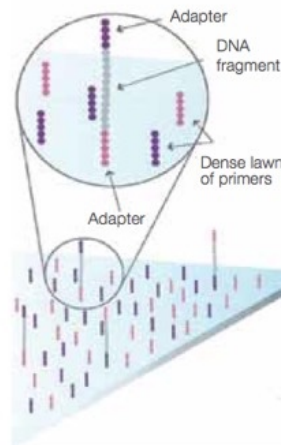
Part I: Recap and Illumina Sequencing

Second Generation Sequencing

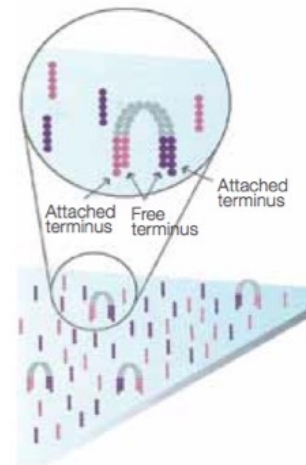


Illumina NovaSeq 6000
Sequencing by Synthesis

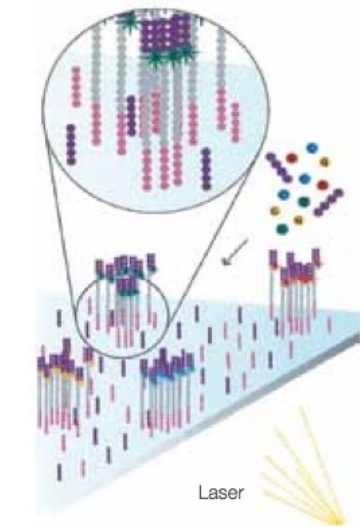
>3Tbp / day
(JHU has 4 of these!)



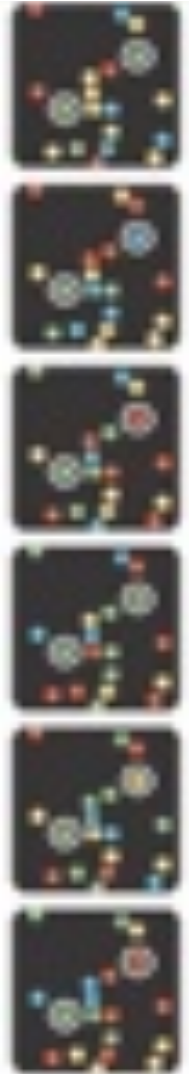
1. Attach



2. Amplify

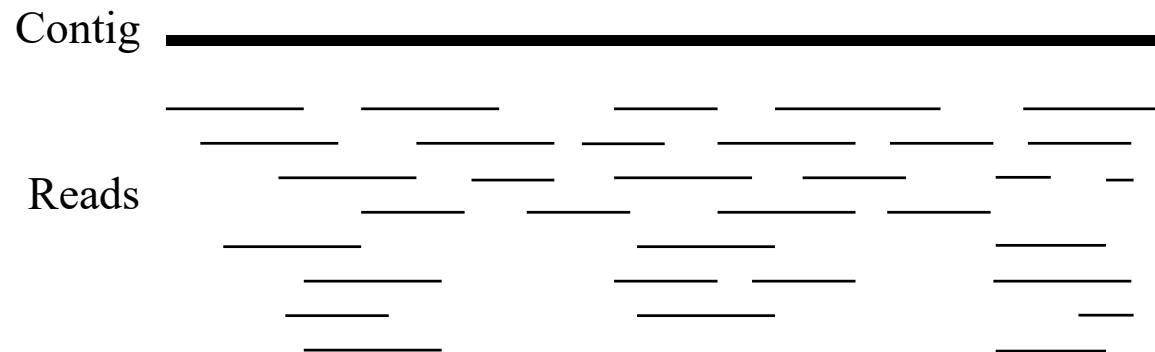
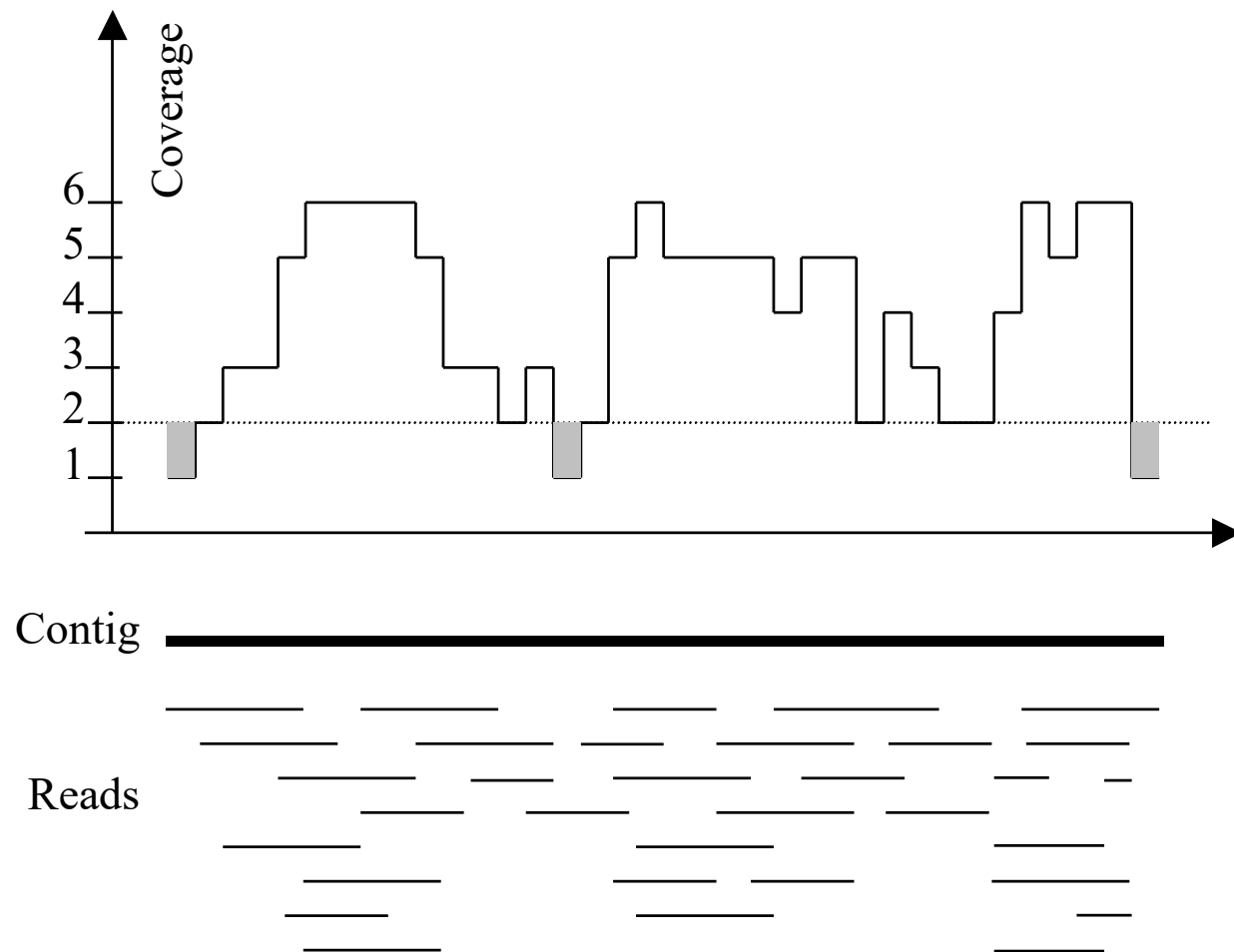


3. Image



Metzker (2010) Nature Reviews Genetics 11:31-46
<https://www.youtube.com/watch?v=fCd6B5HRaZ8>

Typical sequencing coverage

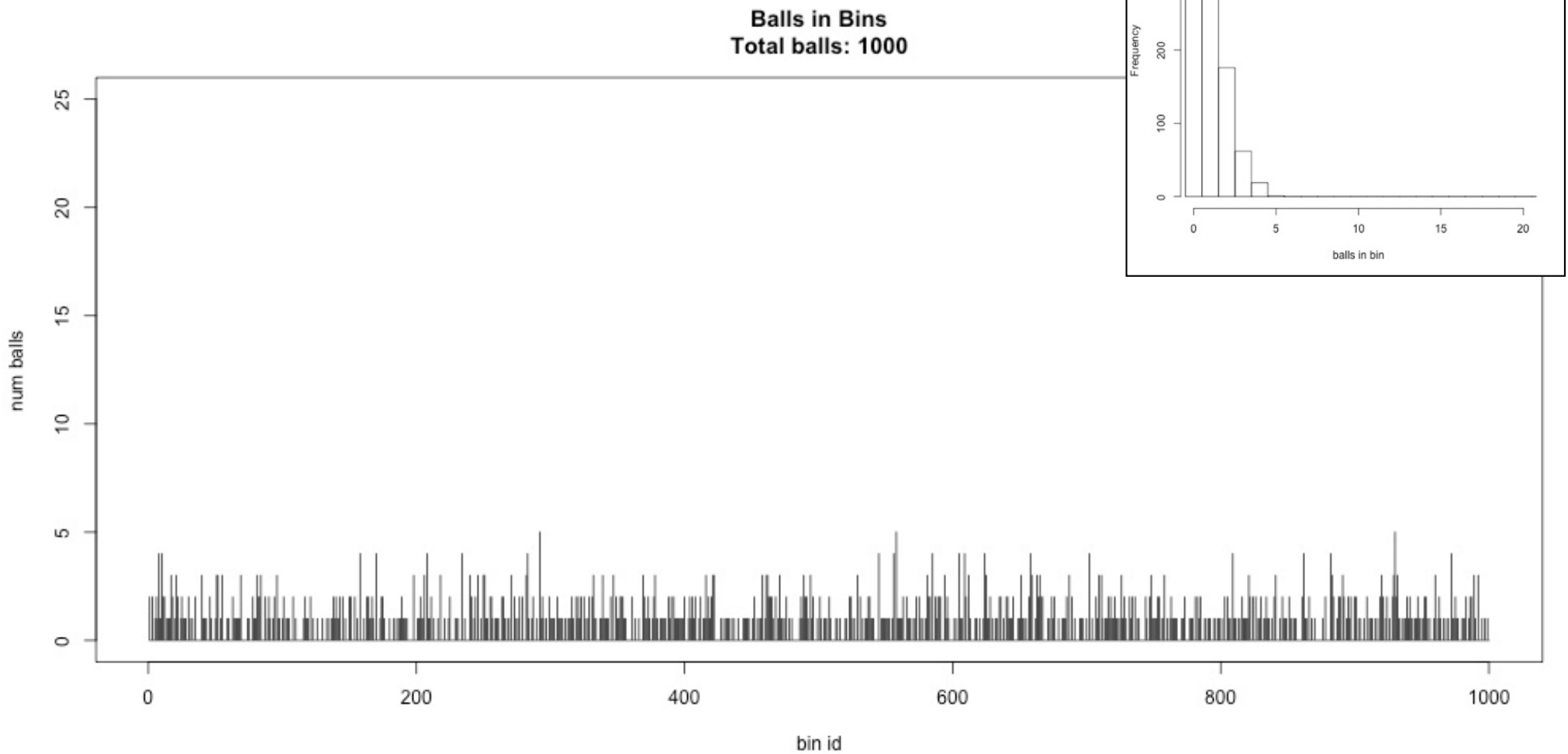


Imagine raindrops on a sidewalk

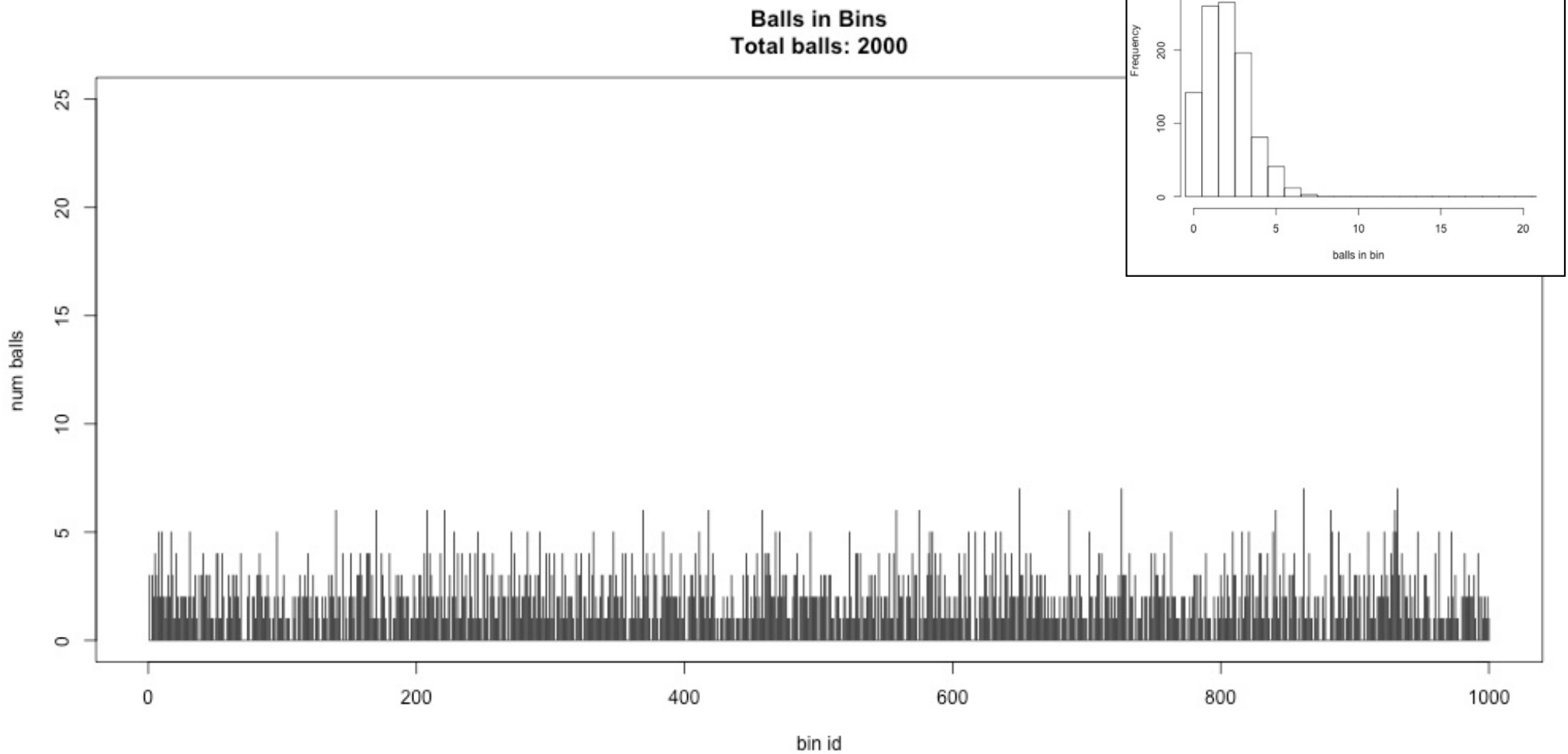
We want to cover the entire sidewalk but each drop costs \$1

If the genome is 10 Mbp, should we sequence 100k 100bp reads?

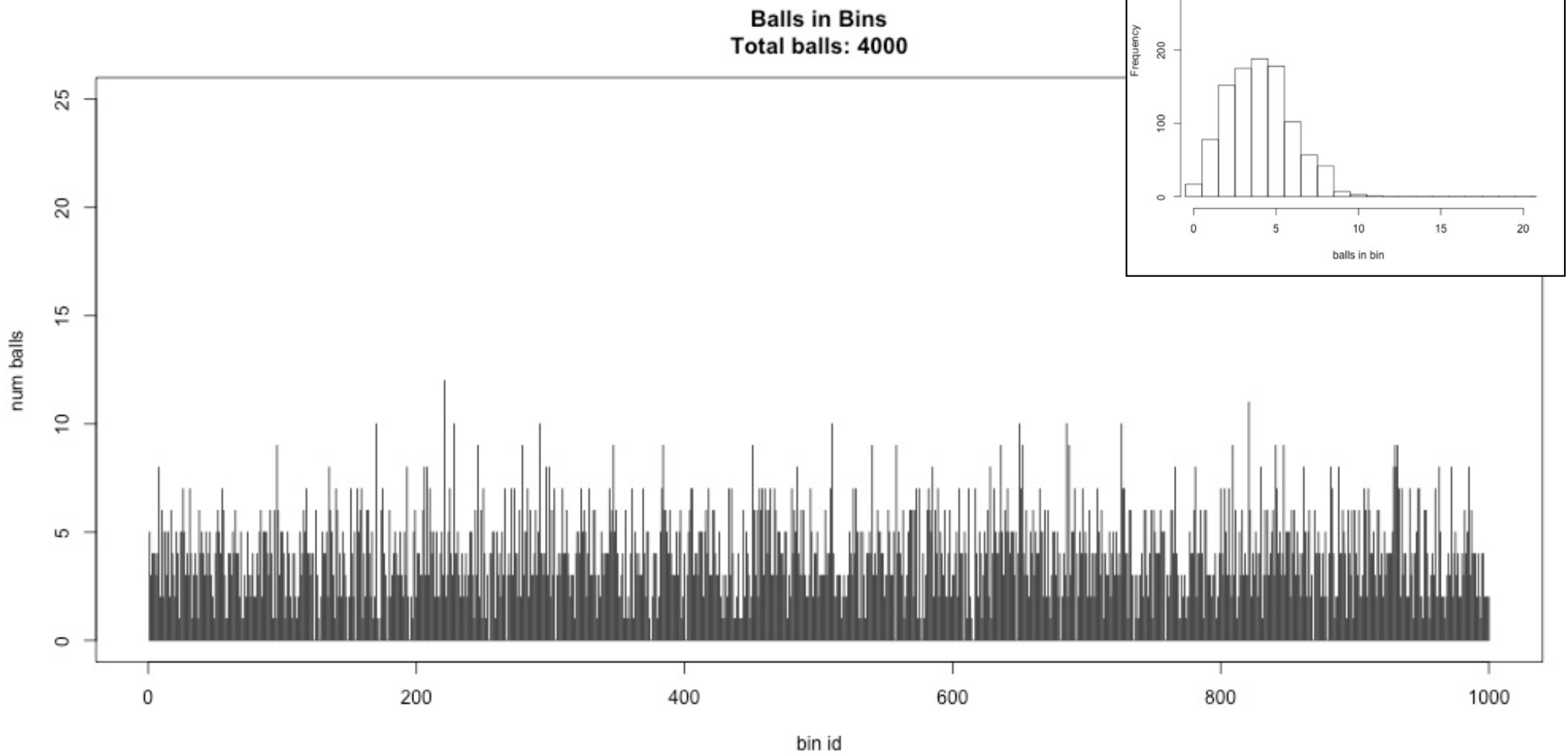
Ix sequencing



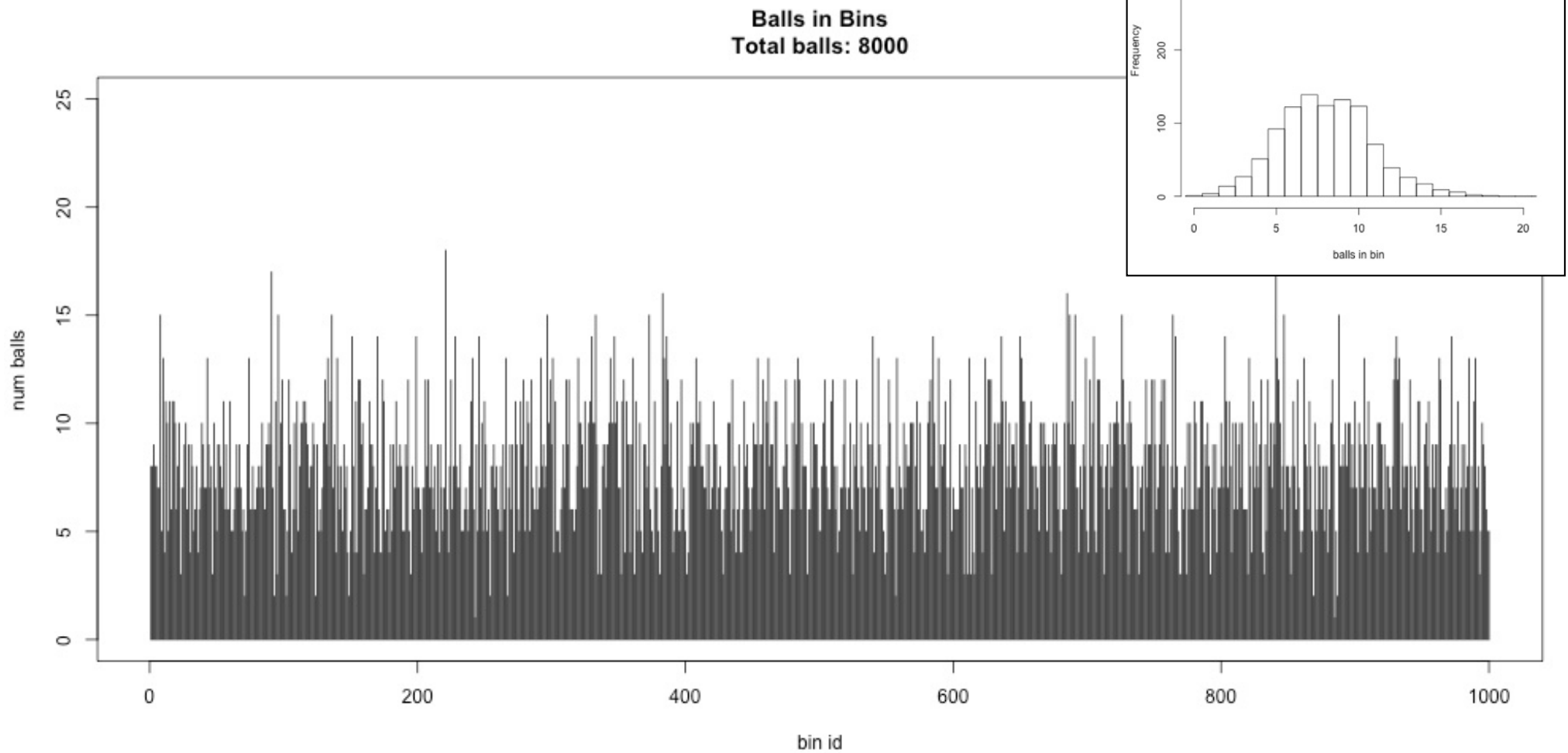
2x sequencing



4x sequencing



8x sequencing



Poisson Distribution

The probability of a given number of events occurring in a fixed interval of time and/or space if these events occur with a known average rate and independently of the time since the last event.

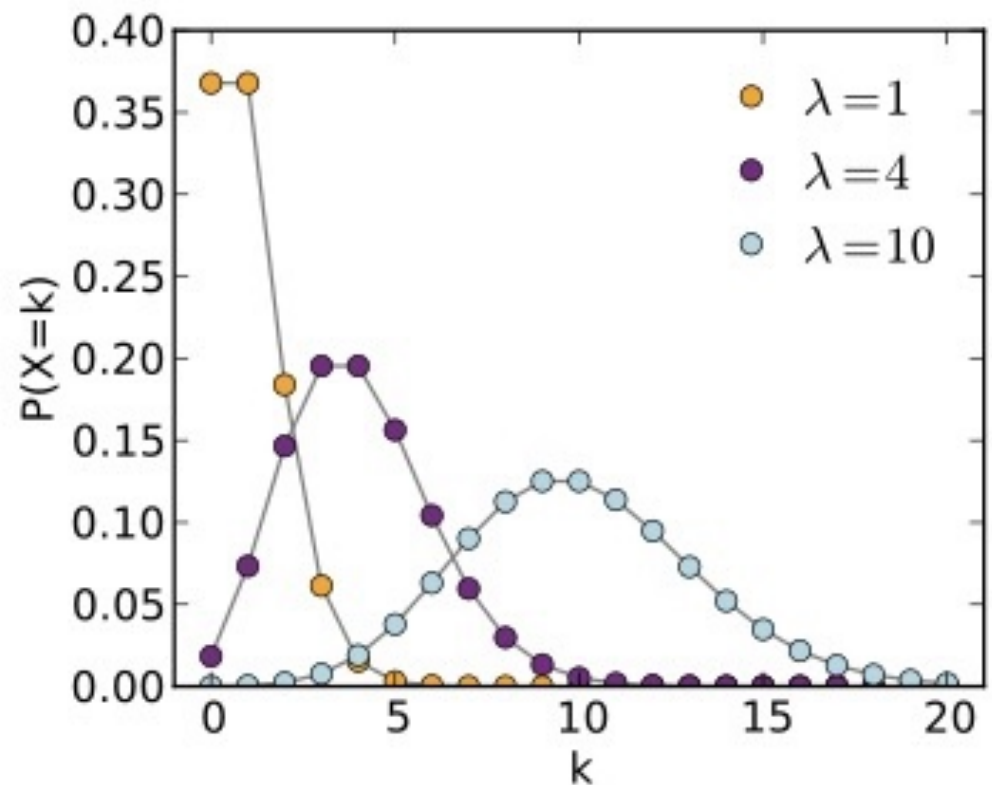
Formulation comes from the limit of the binomial equation

Resembles a normal distribution, but over the positive values, and with only a single parameter.

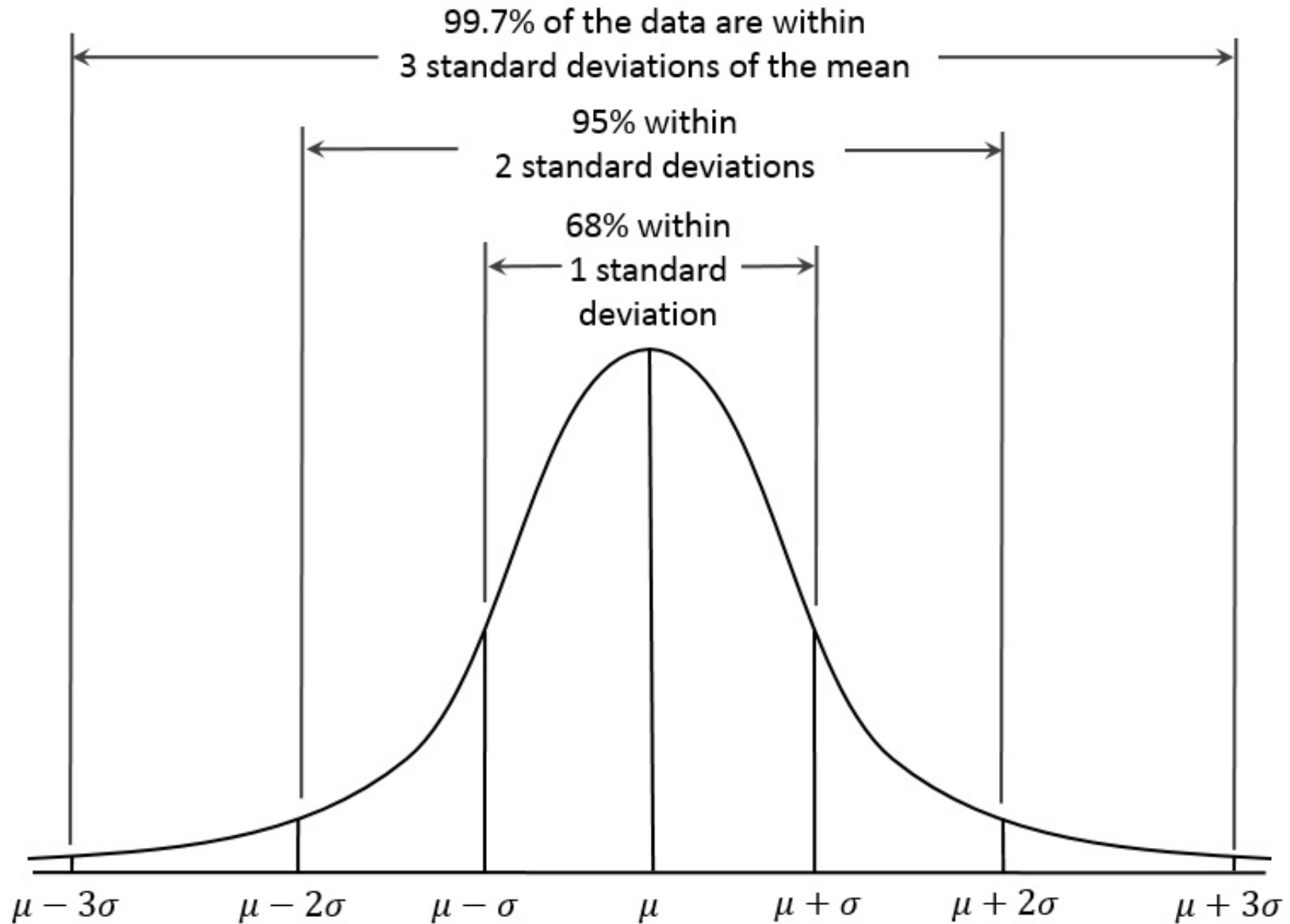
Key properties:

- ***The standard deviation is the square root of the mean.***
- ***For mean > 5, well approximated by a normal distribution***

$$P(k) = \frac{\lambda^k}{k!} e^{-\lambda}$$



Normal Approximation



Can estimate Poisson distribution as a normal distribution when $\lambda > 10$

Pop Quiz!

I want to sequence a 10Mbp genome to 24x coverage.
How many 120bp reads do I need?

I need $10\text{Mbp} \times 24x = 240\text{Mbp}$ of data
 $240\text{Mbp} / 120\text{bp} / \text{read} = 2\text{M}$ reads

I want to sequence a 10Mbp genome so that
>97.5% of the genome has at least 24x coverage.
How many 120bp reads do I need?

Find X such that $X - 2 \times \sqrt{X} = 24$

$$36 - 2 \times \sqrt{36} = 24$$

I need $10\text{Mbp} \times 36x = 360\text{Mbp}$ of data
 $360\text{Mbp} / 120\text{bp} / \text{read} = 3\text{M}$ reads (50% more \$\$\$)

K-mers and K-mer counting

GATTACATACACATTGGATG

GAT ACA ACA ATT GAT

ATT CAT CAC TTG ATG

TTA ATA ACA TGG

TAC TAC CAT GGA

Kmers:

- Divide a string into substrings of length k
- Notice every position is covered k times
- Notice there are $G - k + 1$ kmers from a string of length G
 - 2,999,999,982 19mers in the 3Gb human genome

Computation: Very easy to compute, exact matches, represent 32mers in 64 bits

Biological: The “atomic unit” of a sequence, creates a fingerprint of a genome/read

K-mers and K-mer counting

GATTACATACACATTGGATG

GAT: 2 CAT: 2 ATG: 1 TGG: 1

ACA: 3 CAC: 1 TTA: 1 TAC: 2

ATT: 2 TTG: 1 ATA: 1 GGA: 1

1: 7 (ATG, TGG, ...)

2: 4 (GAT, CAT, ATT, TAC)

3: 1 (ACA)

See HW1

K-mers and K-mer counting

GATTACATACACATTGGATG

1: 7 (ATG, TGG, ...)

2: 4 (GAT, CAT, ATT, TAC)

3: 1 (ACA)

How long should k be?

K-mers and K-mer counting

GATTACATACACATTGGATG

1 : 7 (ATG, TGG, ...)

2 : 4 (GAT, CAT, ATT, TAC)

3 : 1 (ACA)

How long should k be?

K=1 : Too short, every base is present

K=2 : Too short, every pair of bases will be present

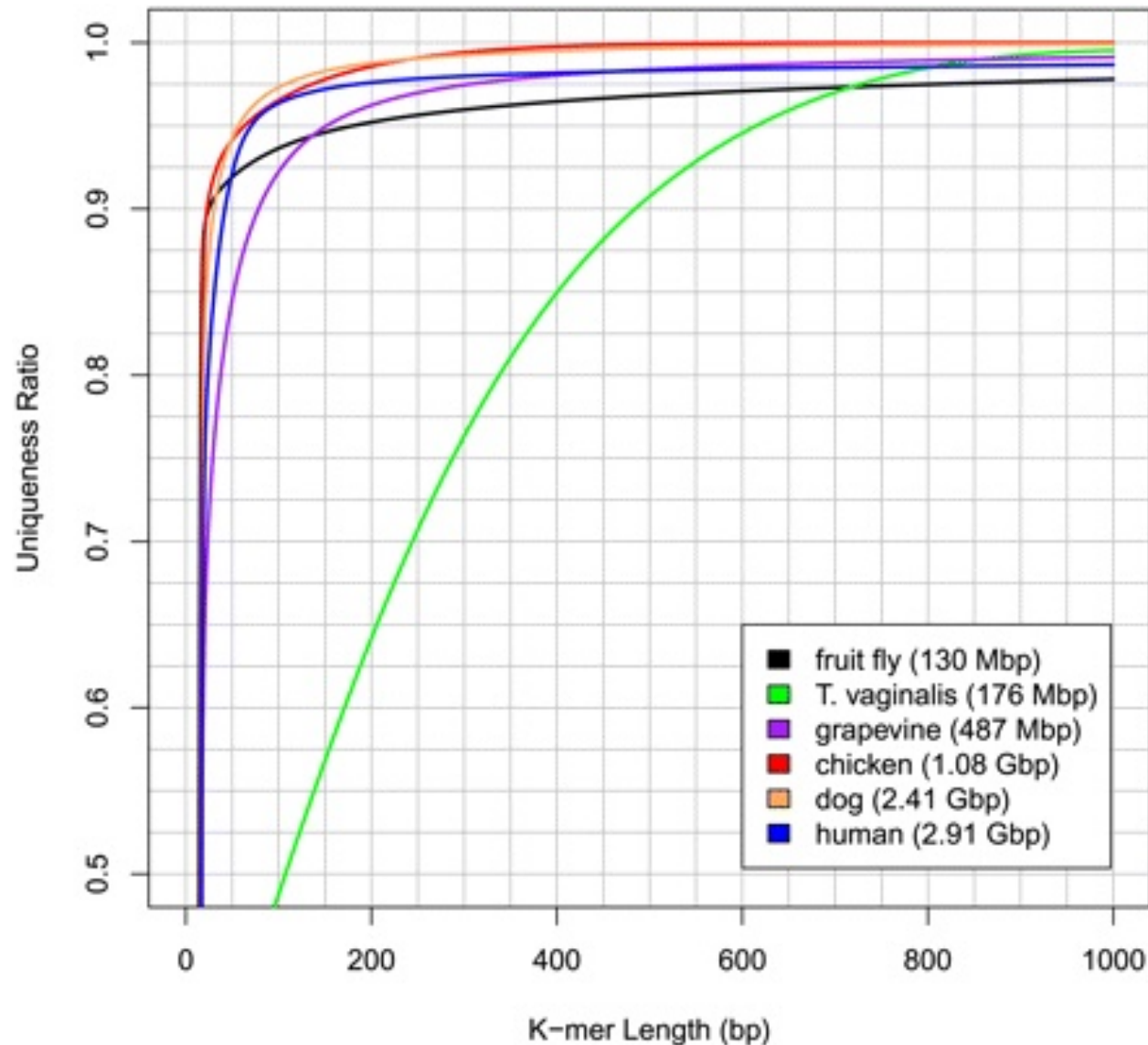
Pick k so that $G/(4^k) \ll 1$

$k = \log_4 (G)$

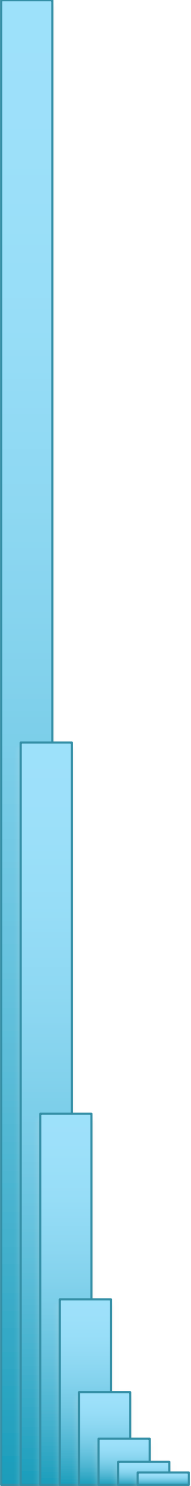
At least 15 for human, often a bit longer

But not too long or could lose resolution

K-mer Uniqueness



Assembly of large genomes using second-generation sequencing
Schatz et al. (2010) Genome Research. doi: 10.1101/gr.101360.109



Part 2: De novo genome assembly



Outline

1. *Assembly theory*

- Assembly by analogy

2. *Practical Issues*

- Coverage, read length, errors, and repeats

3. *Whole Genome Alignment*

- MUMmer recommended

Shredded Book Reconstruction

- Dickens accidentally shreds the first printing of A Tale of Two Cities
 - Text printed on 5 long spools

It was	the best	of times,	it was the	age of wisdom,	it was the	age of foolishness,	...
It was	the best	of times,	it was the	age of wisdom,	it was the	age of foolishness,	...
It was	the best	of times,	it was the	age of wisdom,	it was the	age of foolishness,	...
It was	the best	of times,	it was the	age of wisdom,	it was the	age of foolishness,	...
It	was the best	of times,	it was the	age of wisdom,	it was the	age of foolishness,	...

- How can he reconstruct the text?
 - 5 copies x 138,656 words / 5 words per fragment = 138k fragments
 - The short fragments from every copy are mixed together
 - Some fragments are identical

Greedy Reconstruction

It was the best of
age of wisdom, it was
best of times, it was
it was the age of
it was the age of
it was the worst of
of times, it was the
of times, it was the
of wisdom, it was the
the age of wisdom, it
the best of times, it
the worst of times, it
times, it was the age
times, it was the worst
was the age of wisdom,
was the age of foolishness,
was the best of times,
was the worst of times,
wisdom, it was the age
worst of times, it was

It was the best of
was the best of times,
the best of times, it
best of times, it was
of times, it was the
of times, it was the
times, it was the worst
times, it was the age

The repeated sequence make the correct reconstruction ambiguous

- It was the best of times, it was the [worst/age]

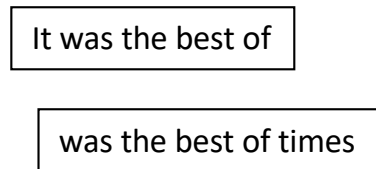
Model the assembly problem as a graph problem

How long will it take to compute the overlaps?

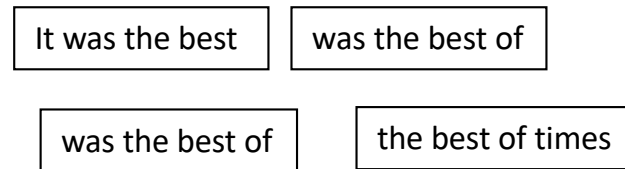
de Bruijn Graph Construction

- $G_k = (V, E)$
 - V = Length- k sub-fragments
 - E = Directed edges between consecutive sub-fragments
 - Sub-fragments overlap by $k-1$ words

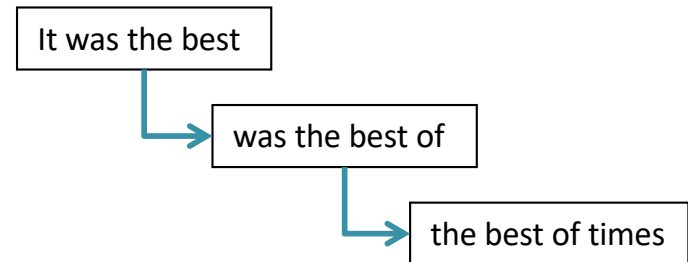
Fragments $|f|=5$



Sub-fragment $k=4$



Directed edges (overlap by $k-1$)



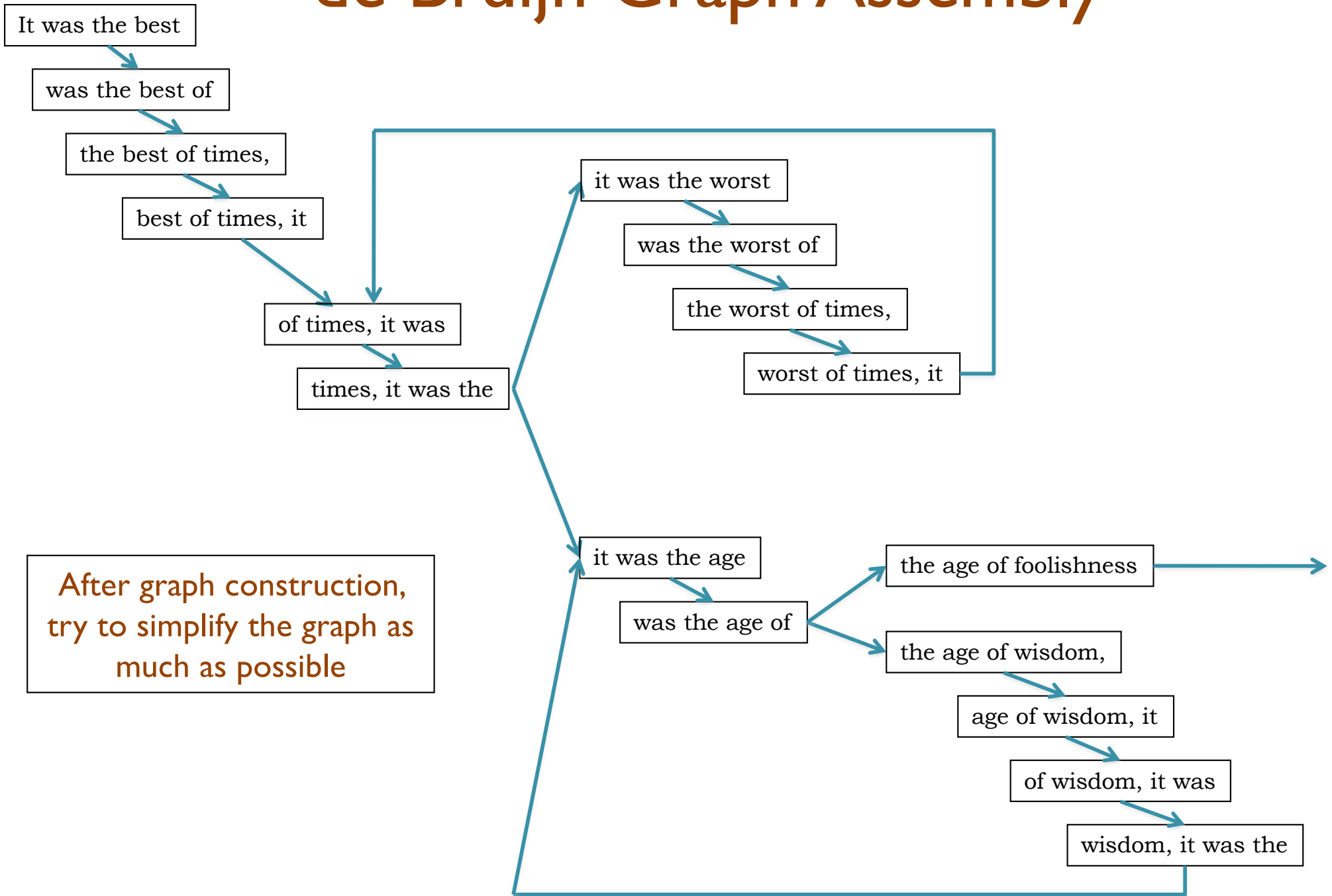
– Overlaps between fragments are implicitly computed

How to pronounce:

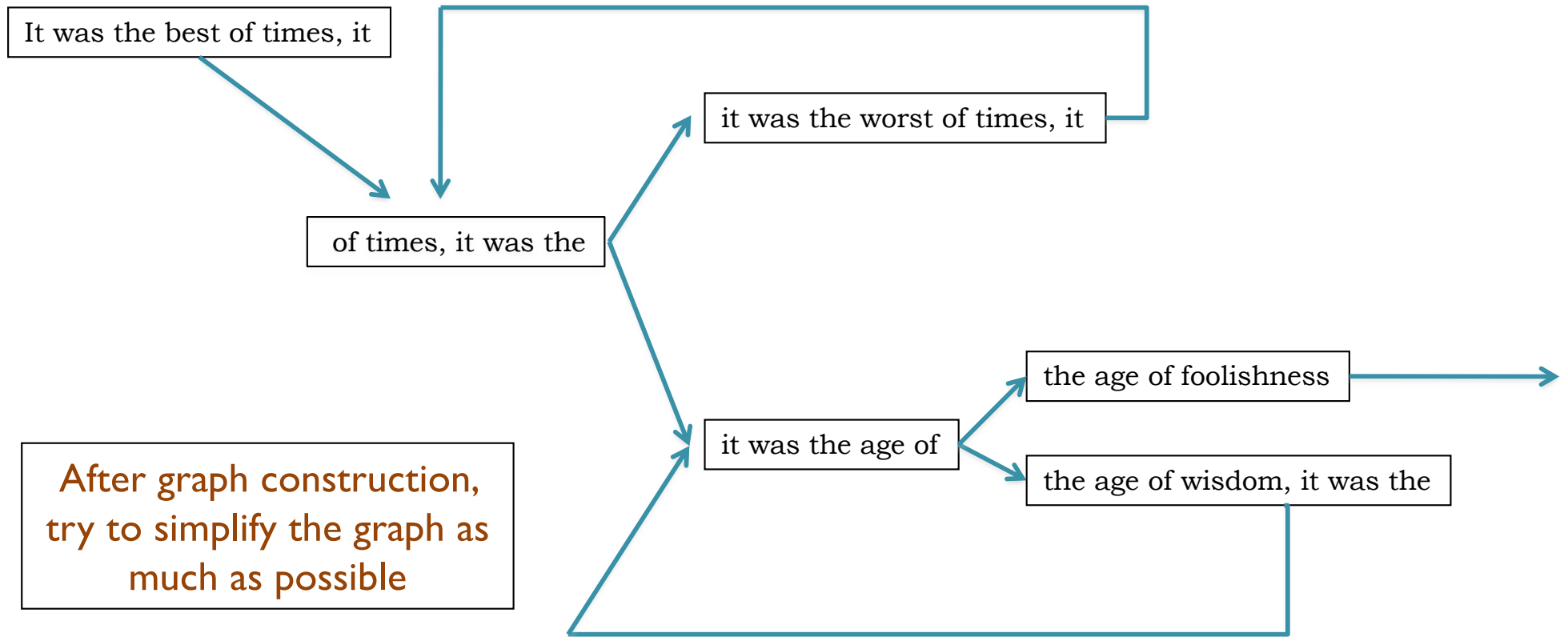
https://forvo.com/word/de_bruijn/

de Bruijn, 1946
Idury et al., 1995
Pevzner et al., 2001

de Bruijn Graph Assembly



de Bruijn Graph Assembly



The full tale

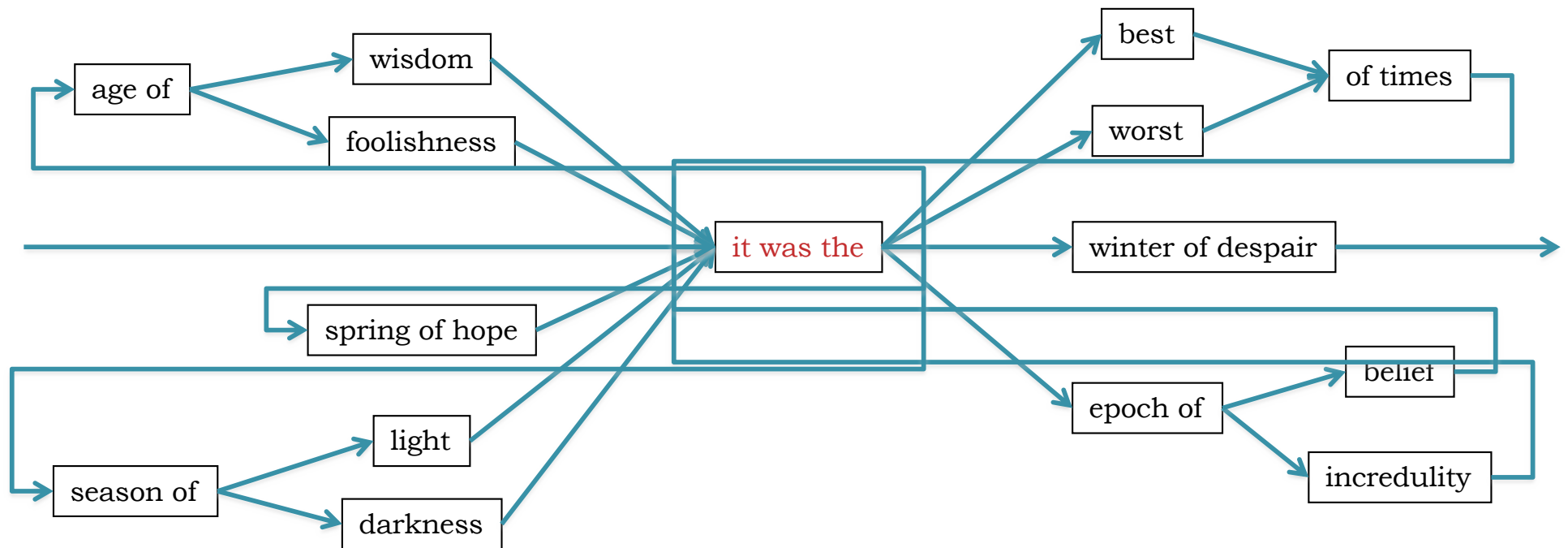
... it was the best of times it was the worst of times ...

... it was the age of wisdom it was the age of foolishness ...

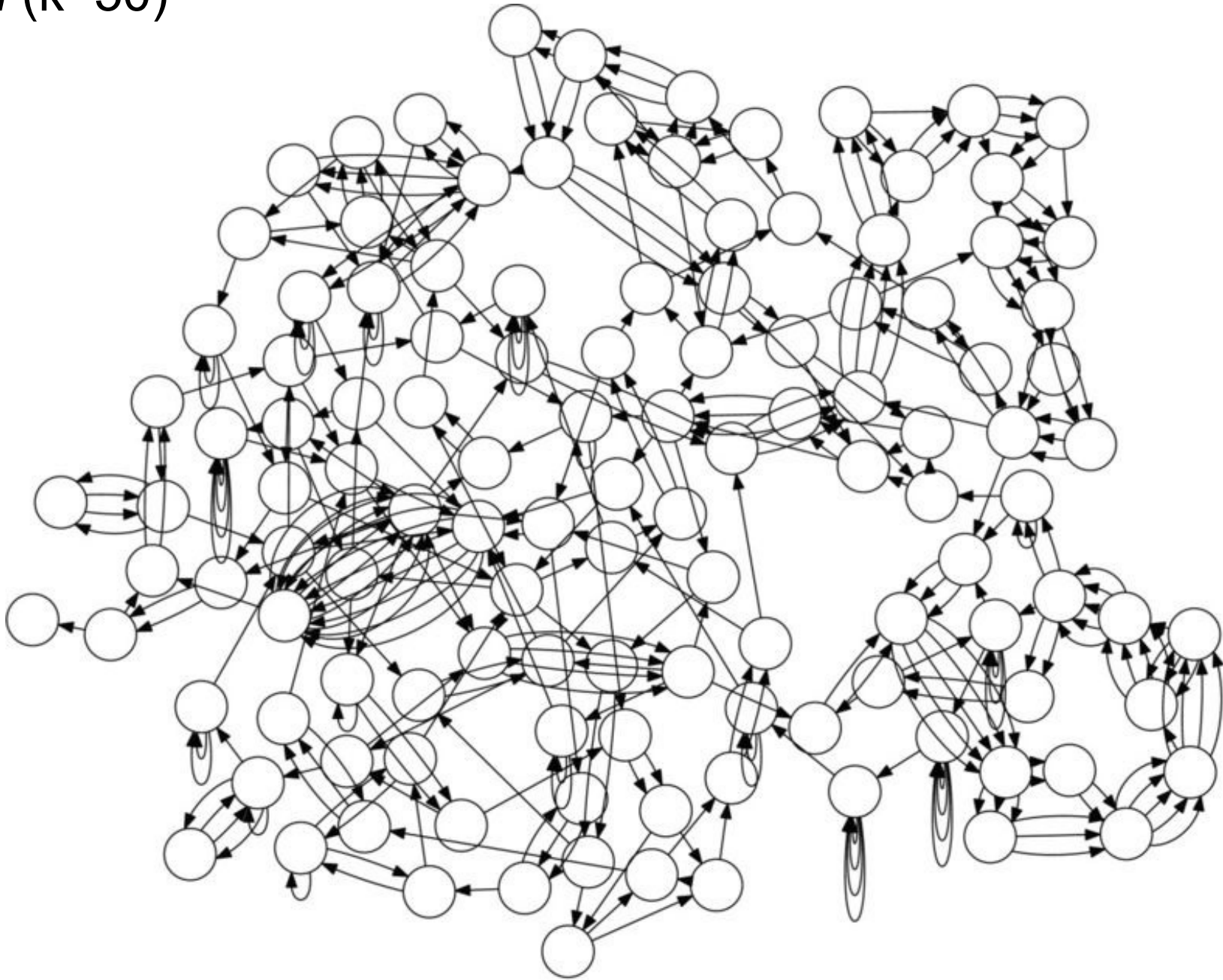
... it was the epoch of belief it was the epoch of incredulity ...

... it was the season of light it was the season of darkness ...

... it was the spring of hope it was the winder of despair ...



E. coli (k=50)



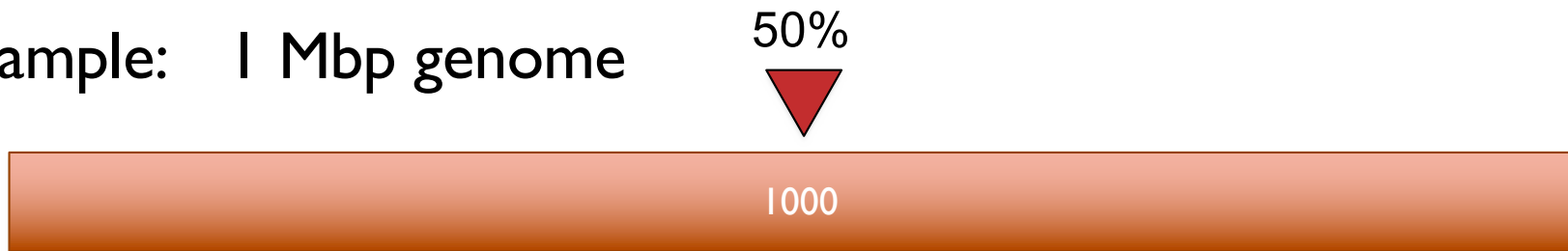
Reducing assembly complexity of microbial genomes with single-molecule sequencing

Koren et al (2013) Genome Biology. 14:R101 <https://doi.org/10.1186/gb-2013-14-9-r101>

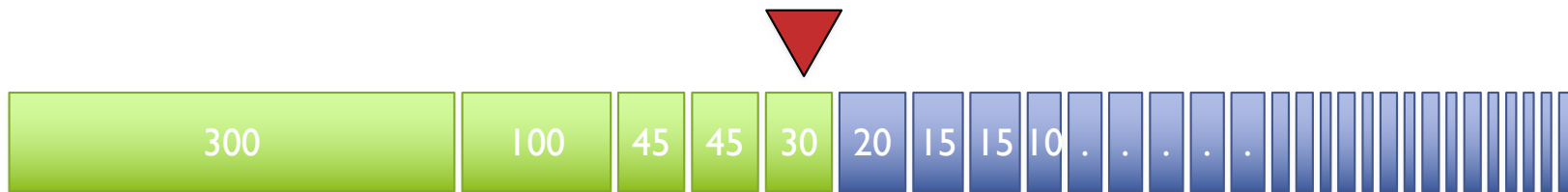
Contig N50

Def: 50% of the genome is in contigs as large as the N50 value

Example: 1 Mbp genome 50%

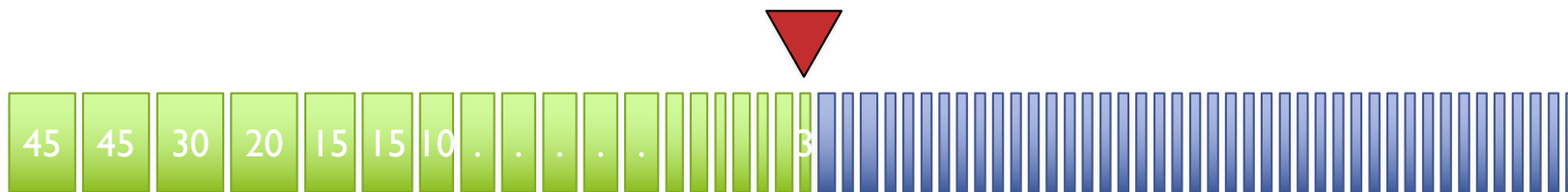



A



N50 size = 30 kbp

B



N50 size = 3 kbp

Contig N50

Def: 50% of the genome is in contigs as large as the N50 value

50%

Better N50s improves the analysis in every dimension

- Better resolution of genes and flanking regulatory regions
- Better resolution of transposons and other complex sequences
- Better resolution of chromosome organization
- Better sequence for all downstream analysis

Just be careful of N50 inflation!

- *A very very very bad assembler in 1 line of bash:*
- *cat *.reads.fa > genome.fa*

N50 size = 3 kbp

Pop Quiz I

Assemble these reads using a de Bruijn graph approach ($k=3$):

ACACG

ATTAC

GATTA

TTACA

Pop Quiz I

Assemble these reads using a de Bruijn graph approach (k=3):

ACACG : ACA → CAC → ACG
ATTAC : ATT → TTA → TAC
GATTA : GAT → ATT → TTA
TTACA : TTA → TAC → ACA

Pop Quiz I

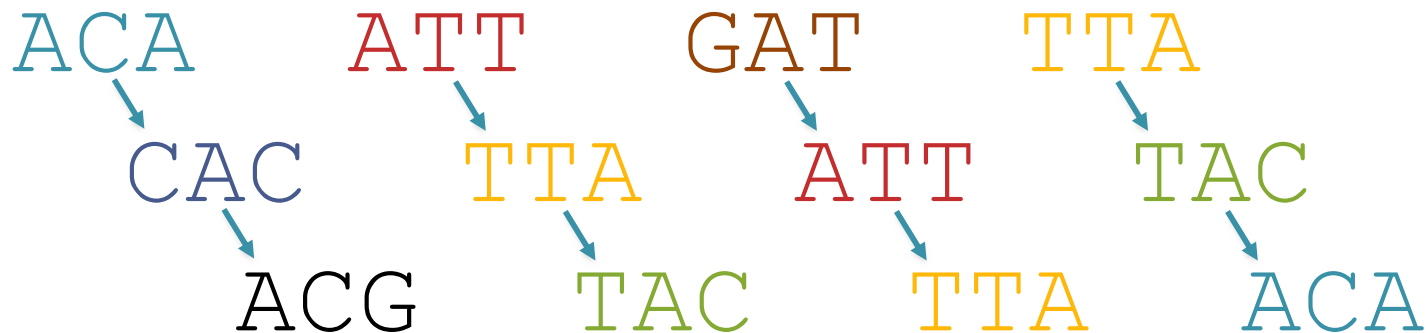
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG: ACA → CAC → ACG

ATTAC: ATT → TTA → TAC

GATTA: GAT → ATT → TTA

TTACA: TTA → TAC → ACA



Pop Quiz I

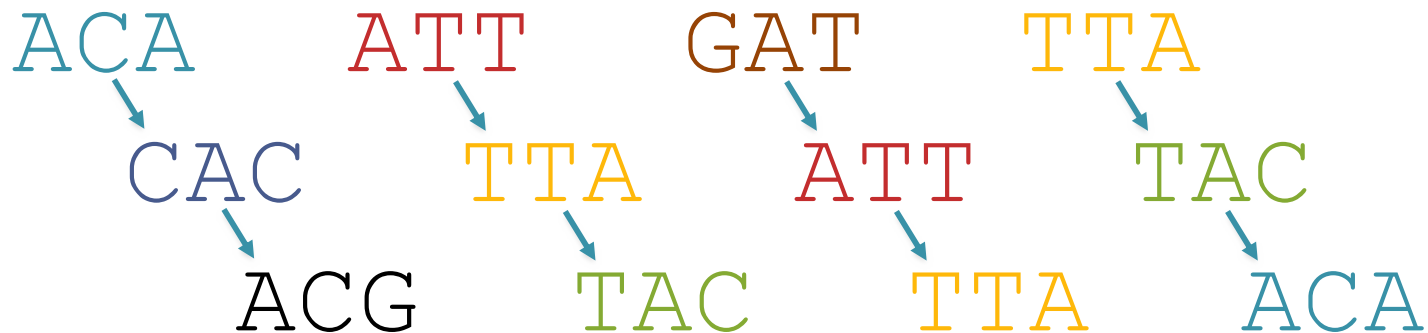
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

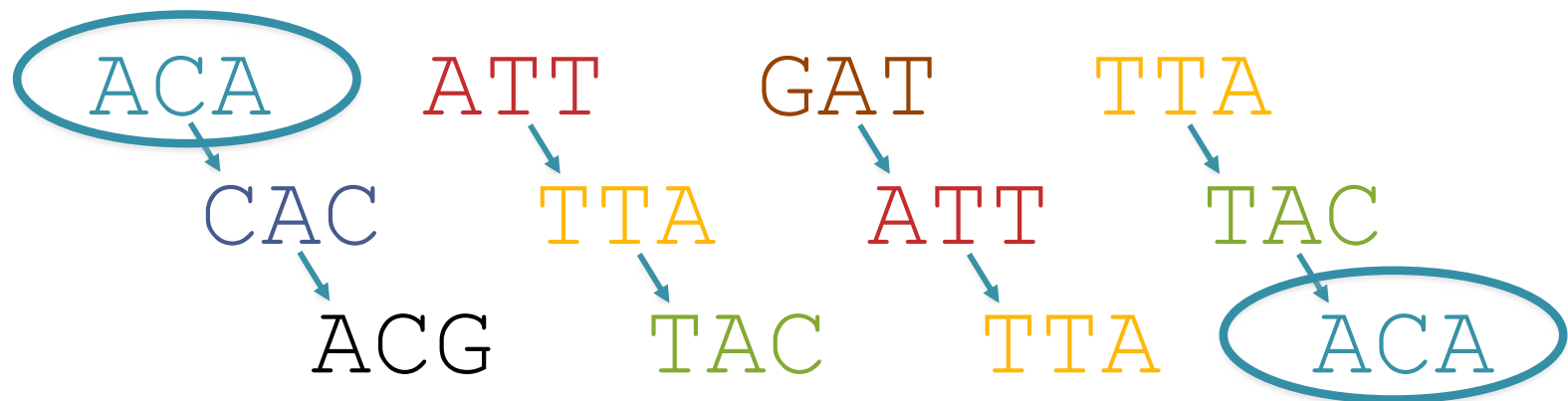
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

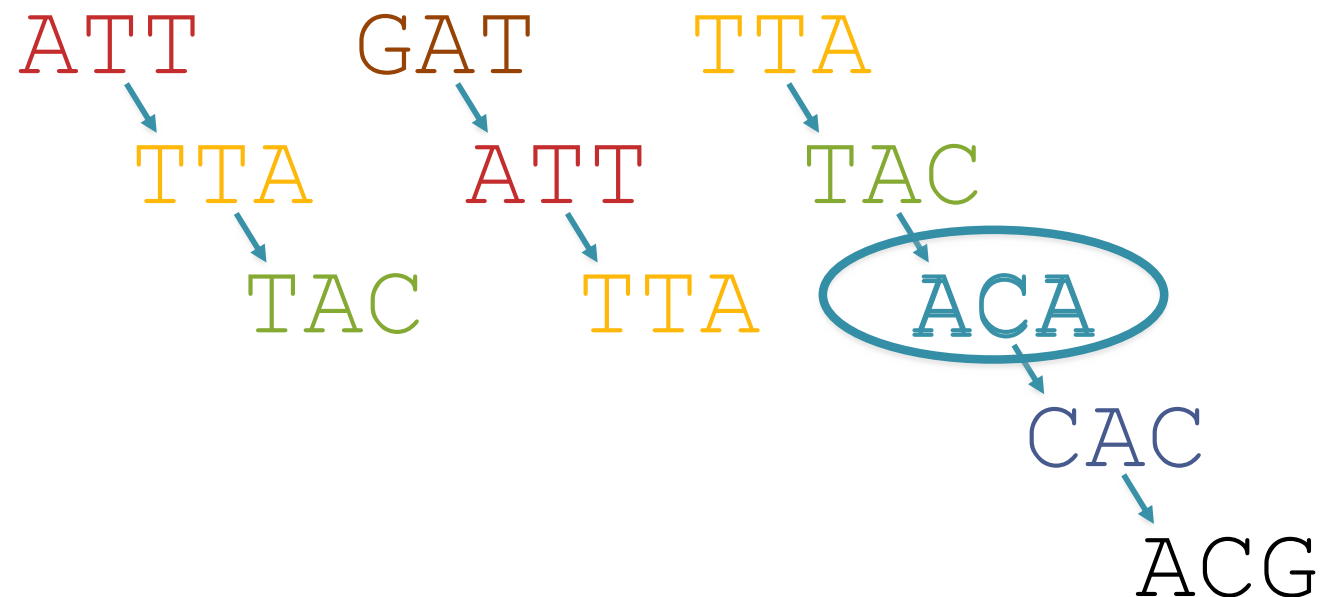
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

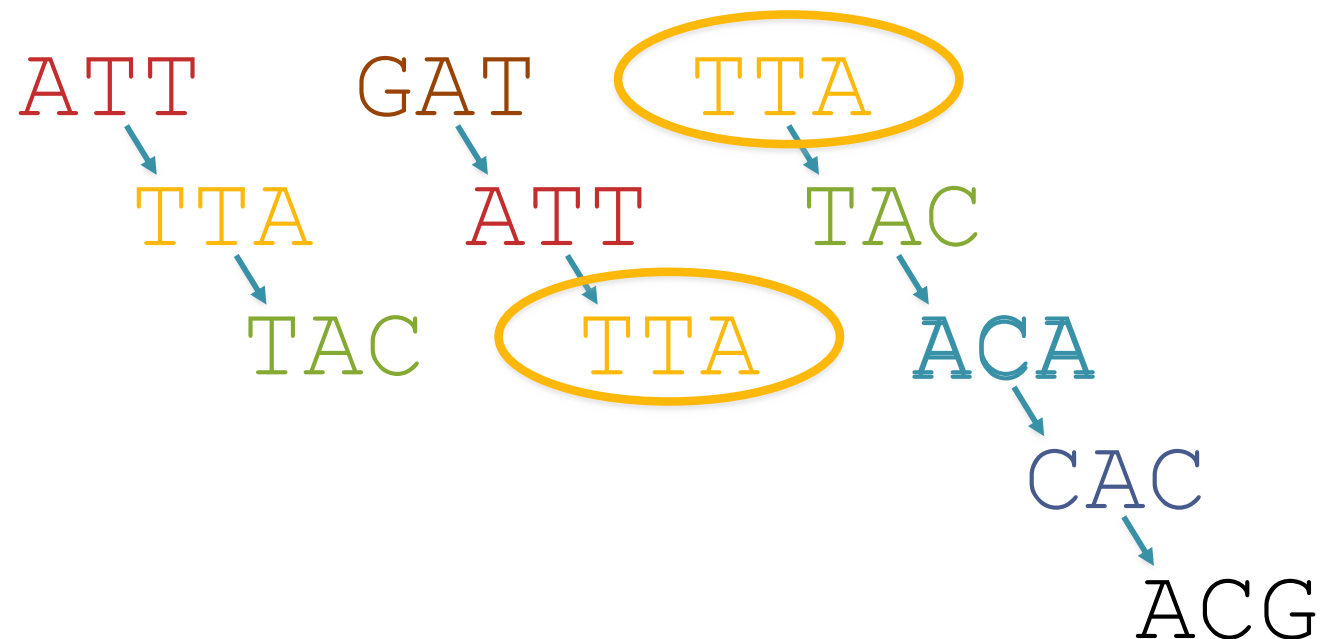
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

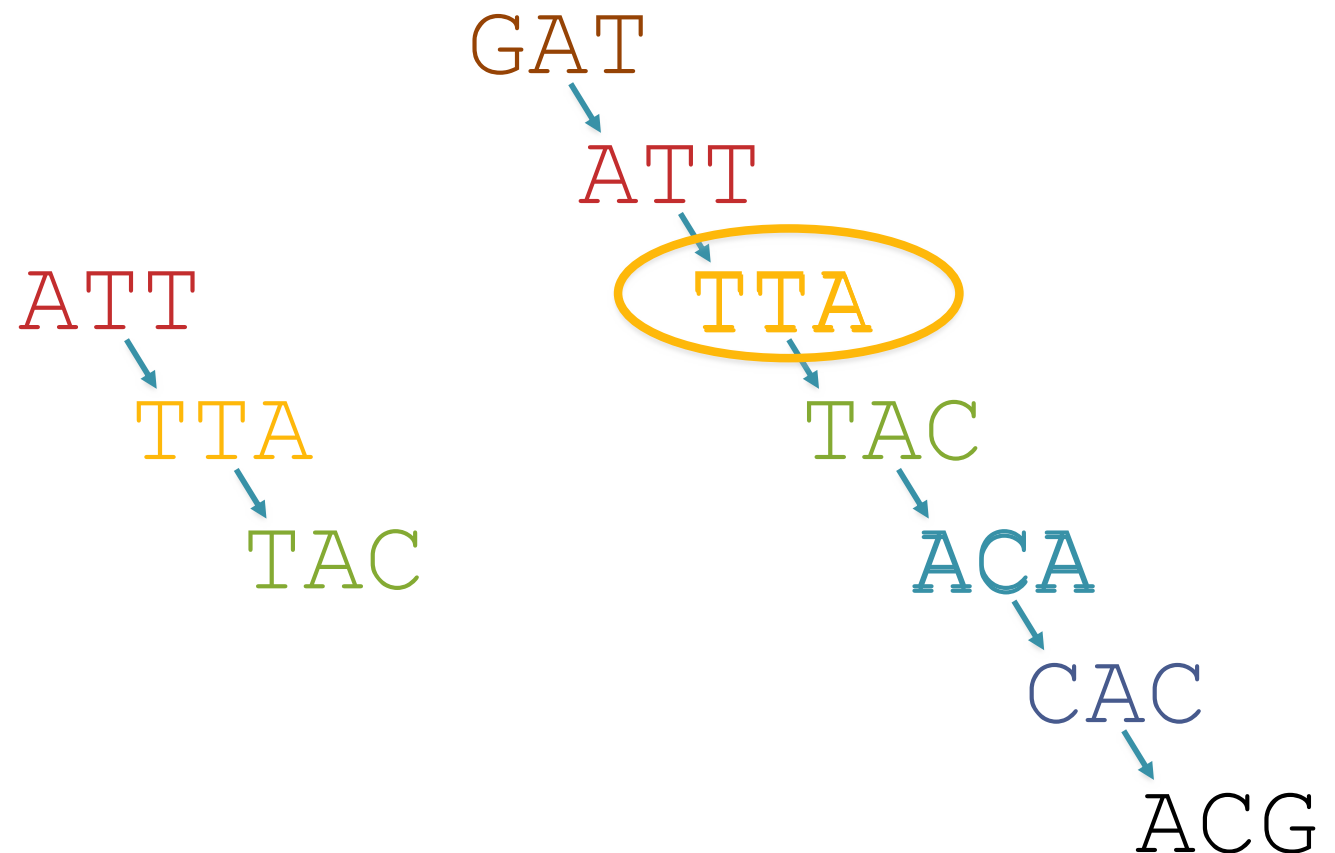
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

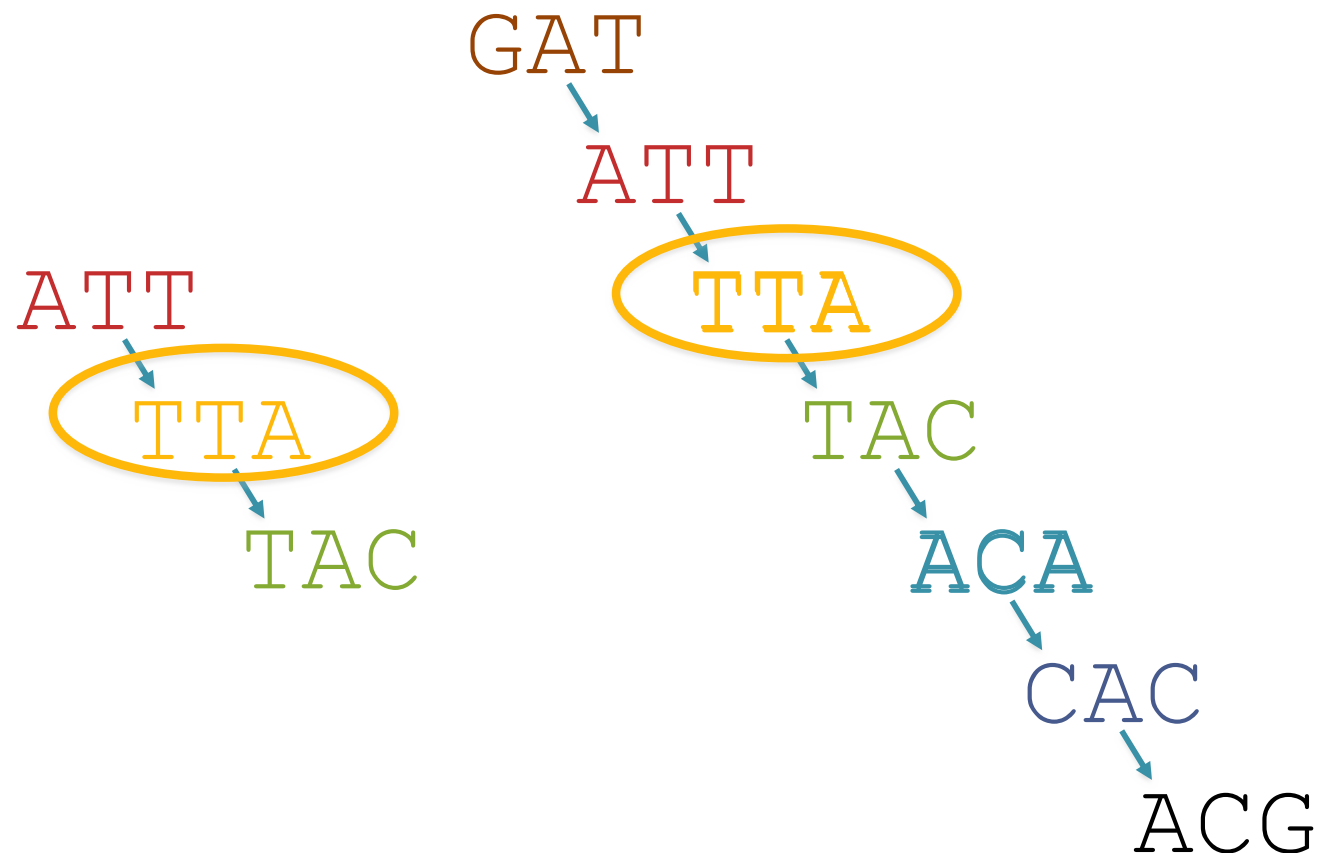
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

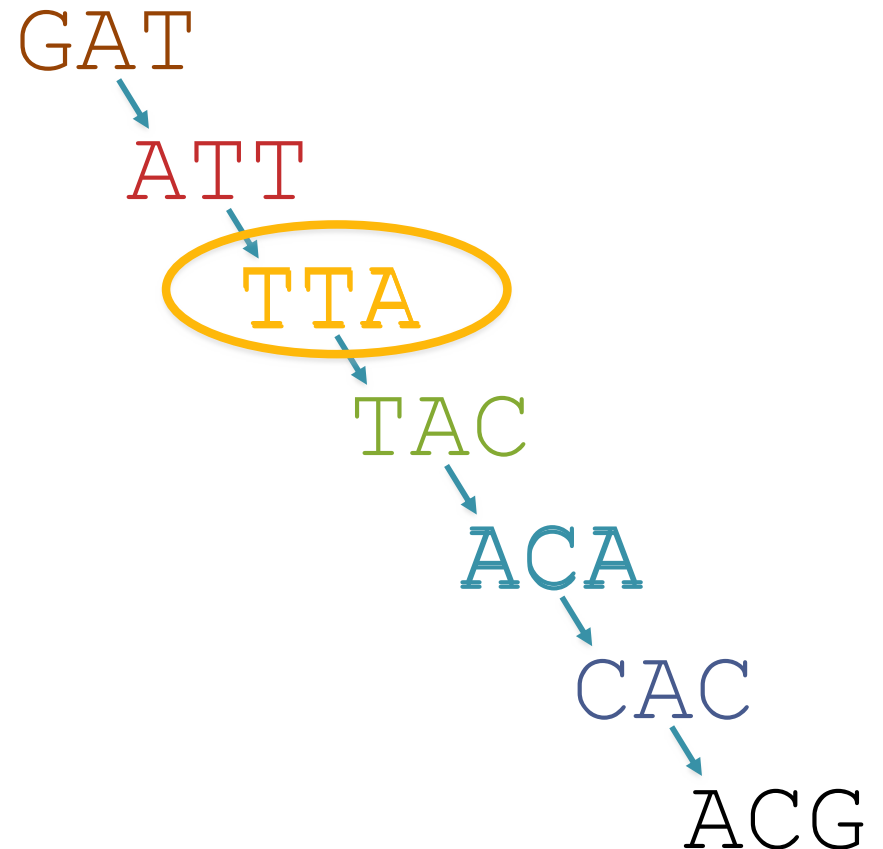
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

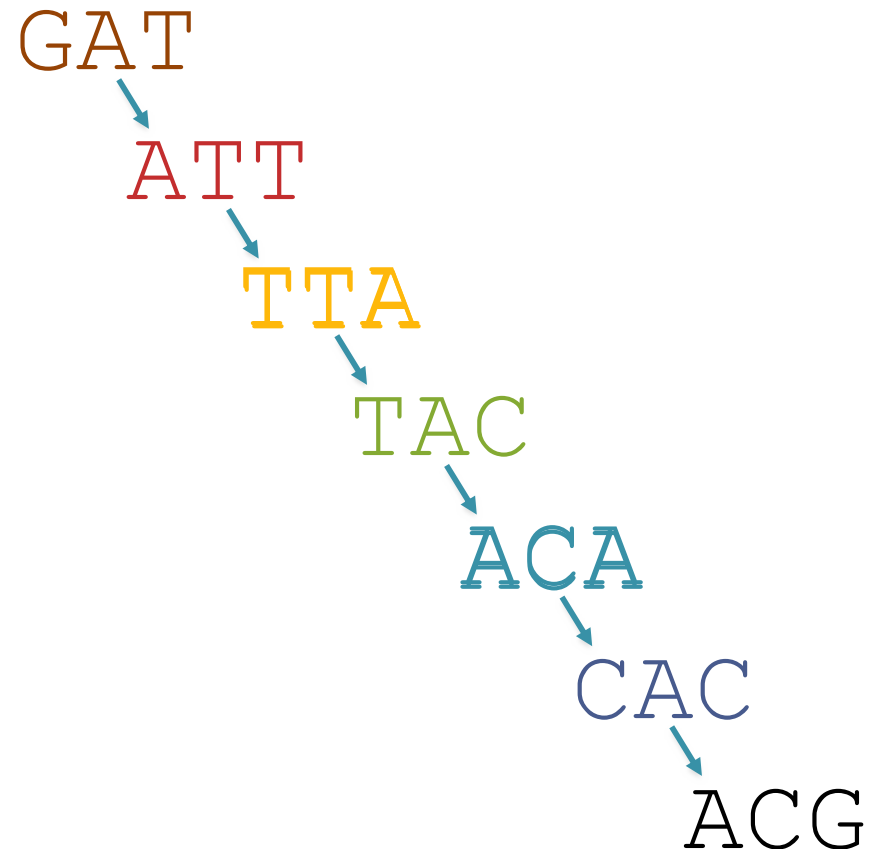
Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA



Pop Quiz I

Assemble these reads using a de Bruijn graph approach (k=3):

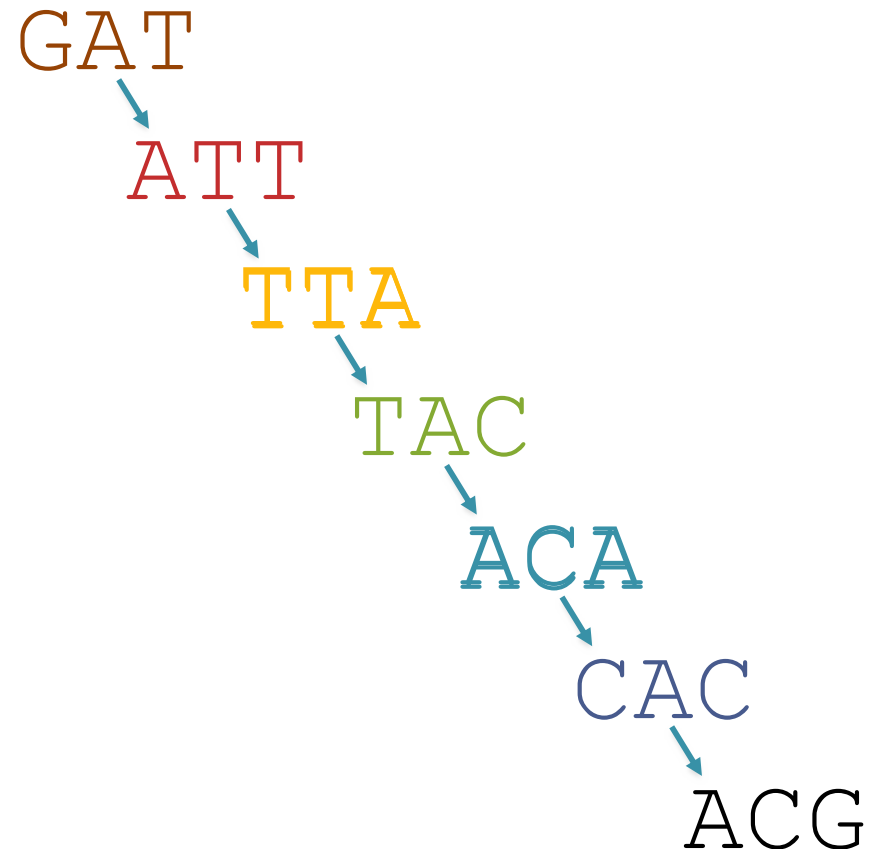
ACACG

ATTAC

GATTA

TTACA

GATTACACG



Pop Quiz I

Assemble these reads using a de Bruijn graph approach (k=3):

ACACG

ATTAC

GATTA

TTACA

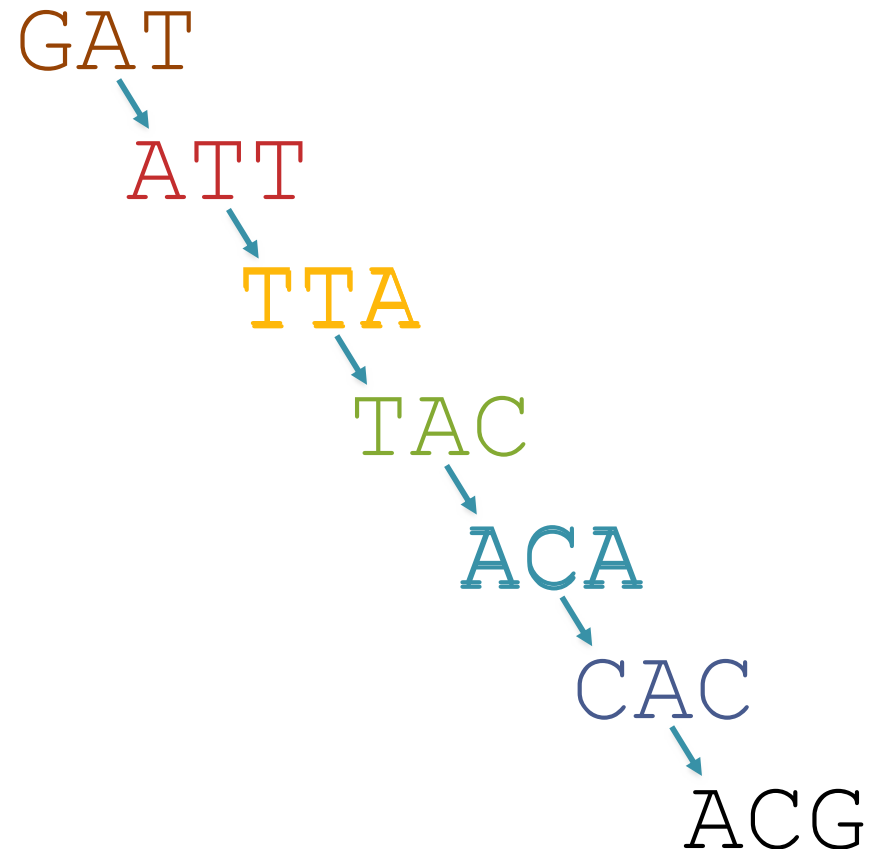
GATTACACG

GATTA

ATTAC

TTACA

ACACG



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach ($k=3$):

ACGA

ACGT

ATAC

CGAC

CGTA

GACG

GTAT

TACG

Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

~~ACGA~~

ACGT

ATAC

CGAC

CGTA

GACG

GTAT

TACG

ACG → CGA

Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

~~ACGA~~

~~ACGT~~

ATAC

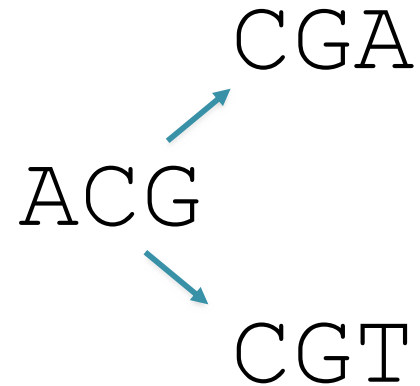
CGAC

CGTA

GACG

GTAT

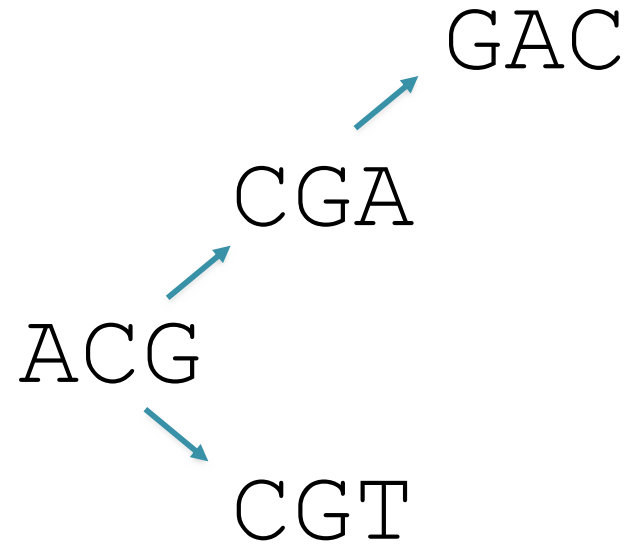
TACG



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

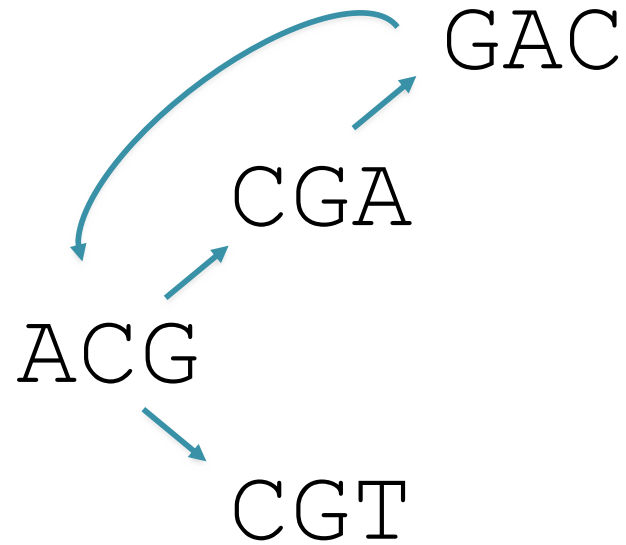
~~ACGA~~
~~ACGT~~
ATAC
~~CGAC~~
CGTA
GACG
GTAT
TACG



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

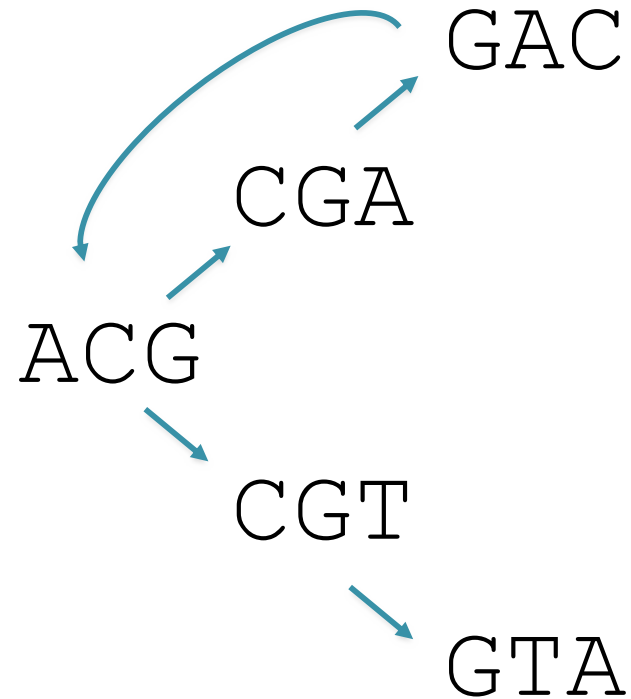
~~ACGA~~
~~ACGT~~
ATAC
~~CGAC~~
CGTA
~~GACG~~
GTAT
TACG



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

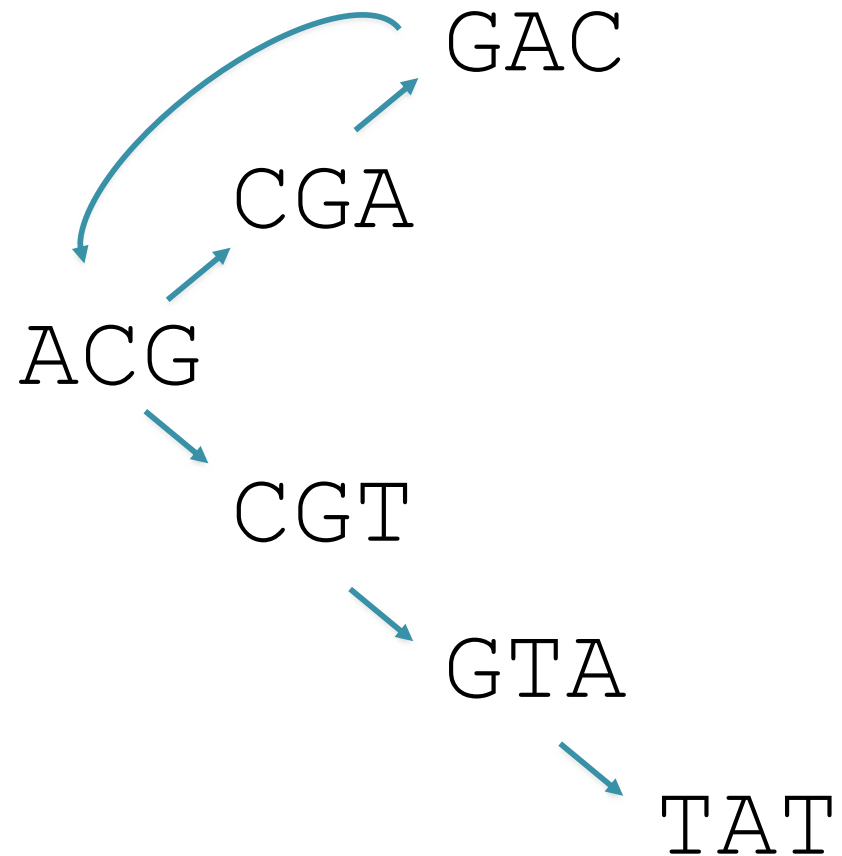
~~ACGA~~
~~ACGT~~
ATAC
~~CGAC~~
~~CGTA~~
~~GACG~~
GTAT
TACG



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

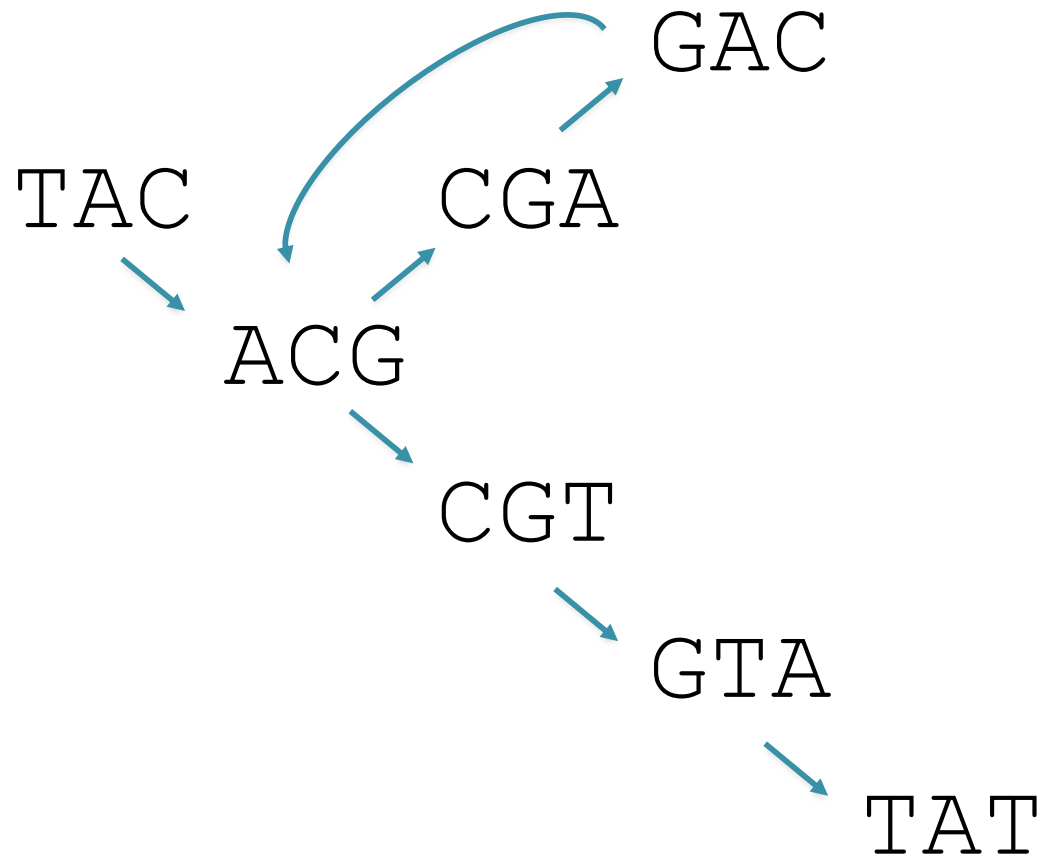
~~ACGA~~
~~ACGT~~
ATAC
~~CGAC~~
~~CGTA~~
~~GACG~~
~~GTAT~~
TACG



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

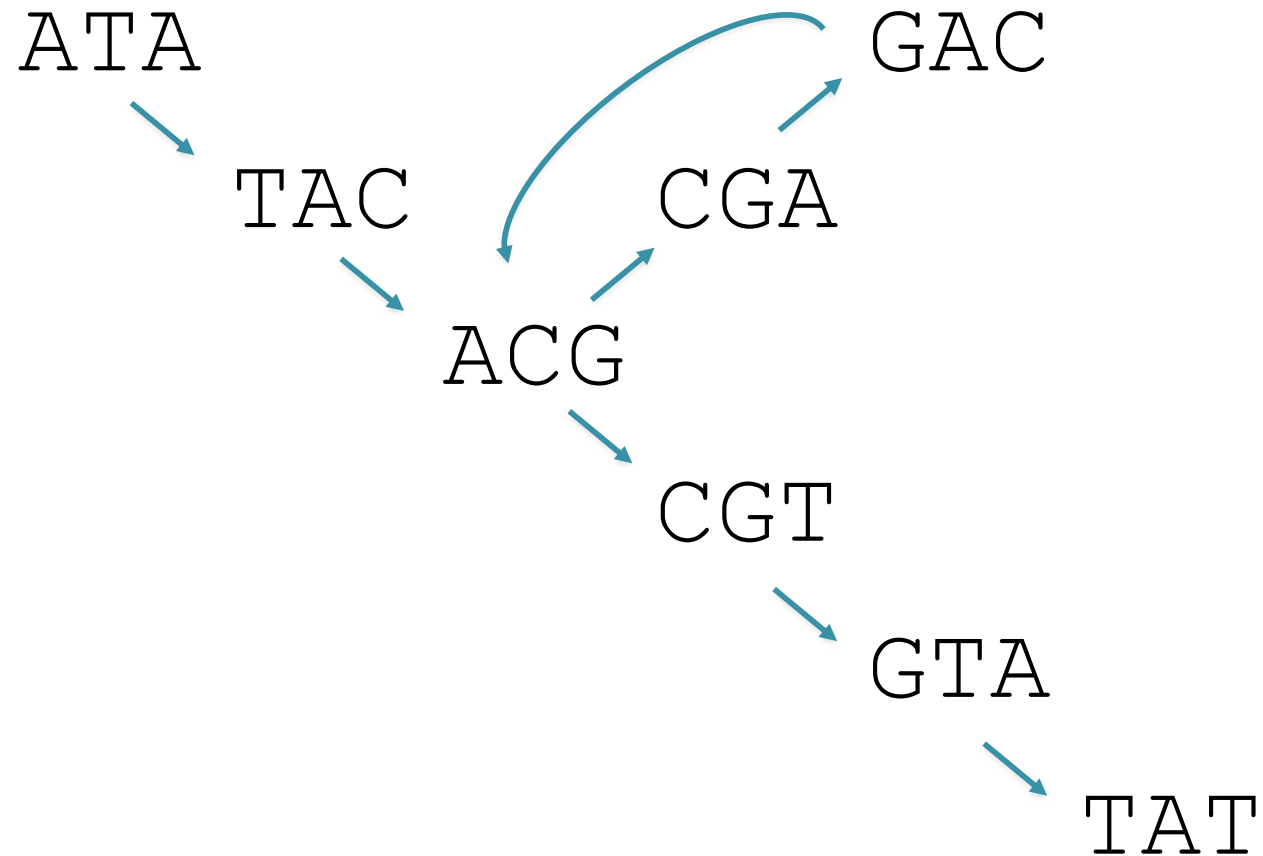
~~ACGA~~
~~ACGT~~
ATAC
~~CGAC~~
~~CGTA~~
~~GACG~~
~~GTAT~~
~~TACG~~



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

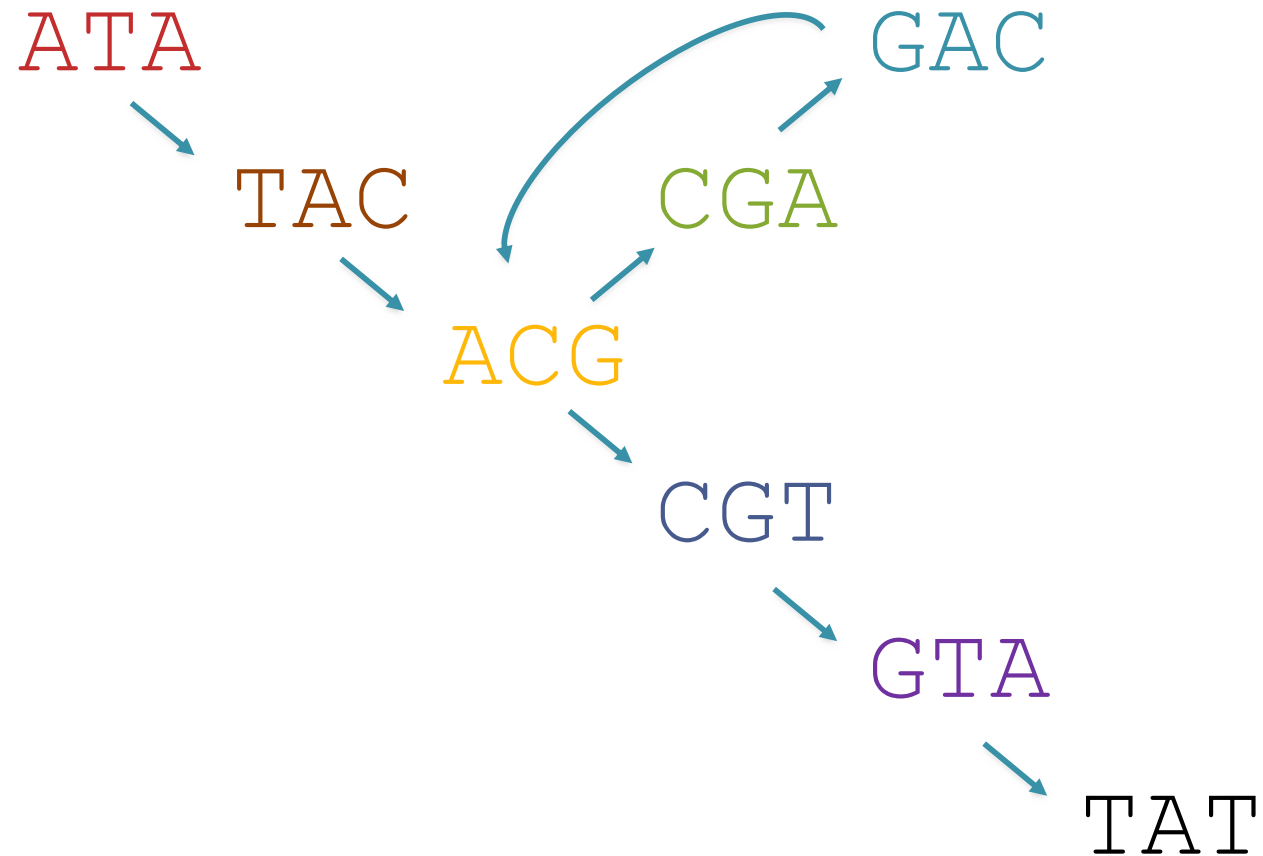
~~ACGA~~
~~ACGT~~
~~ATAC~~
~~CGAC~~
~~CGTA~~
~~GACG~~
~~GTAT~~
~~TACG~~



Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

~~ACGA~~
~~ACGT~~
~~ATAC~~
~~CGAC~~
~~CGTA~~
~~GACG~~
~~GTAT~~
~~TACG~~

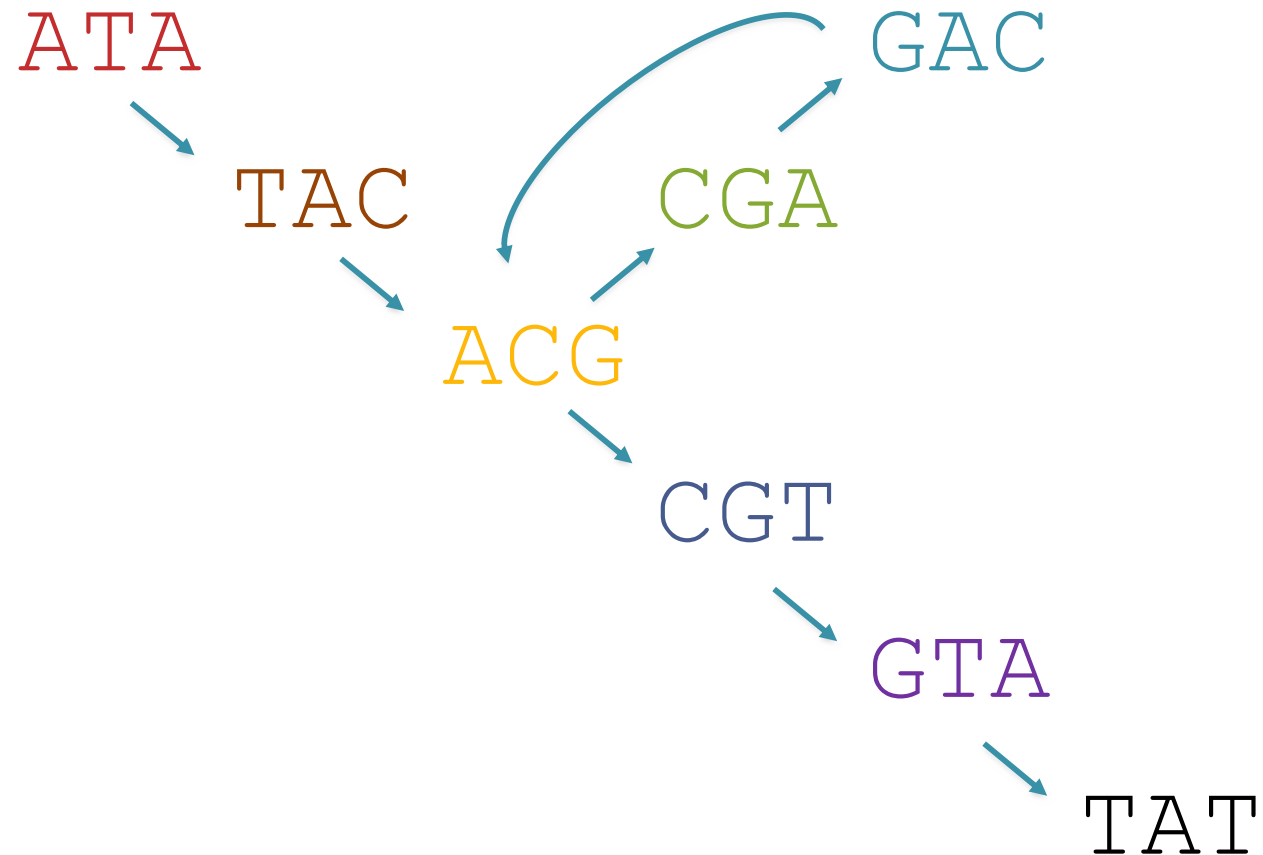


ATACGACGTAT

Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

~~ACGA~~
~~ACGT~~
~~ATAC~~
~~CGAC~~
~~CGTA~~
~~GACG~~
~~GTAT~~
~~TACG~~



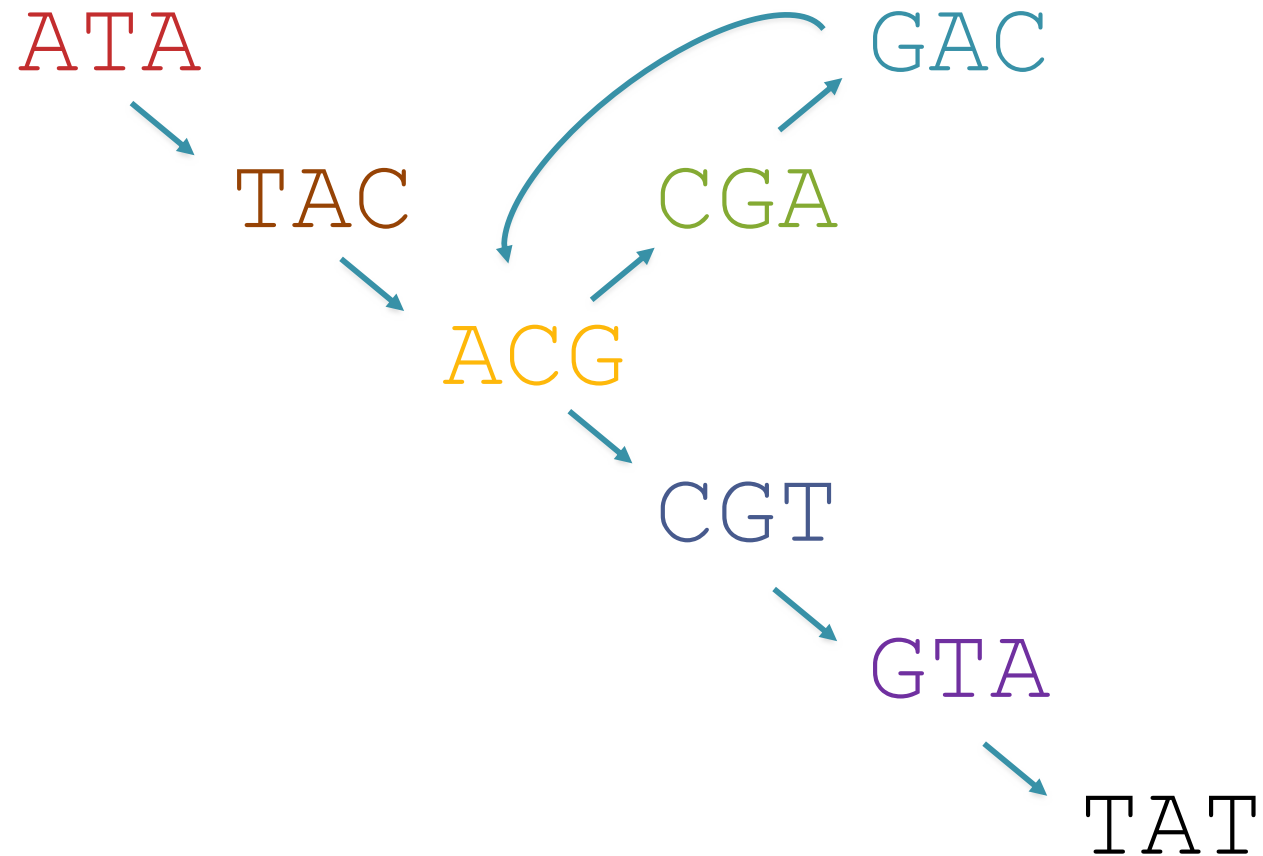
Whats another possible genome?

ATACGACGTAT

Pop Quiz 2

Assemble these reads using a de Bruijn graph approach (k=3):

~~ACGA~~
~~ACGT~~
~~ATAC~~
~~CGAC~~
~~CGTA~~
~~GACG~~
~~GTAT~~
~~TACG~~



Should we add the edge TAT -> ATA?

ATACGACGTAT